

FRONTEND · ARCHITECTURE

AL-AZHARI LIBRARY



FRONTEND IMPLEMENTATION PLAN

Angular frontend · Arabic-first / RTL-first

Version 1.0

PREPARED 4 October 2026

Contents

AL-AZHARI LIBRARY — مكتبة الأزهرى	4
FRONTEND IMPLEMENTATION PLAN	4
1. Executive Summary	4
2. Frontend Goals	5
3. Source Documents & Authority	6
4. Existing System Analysis	6
4.1 Backend (consumed, not rebuilt)	6
4.2 Endpoints the backend plan specifies (\$48 of the backend plan)	7
4.3 Gaps found, and why they matter (read before starting)	7
5. Frontend Architecture	9
5.1 Layered model	9
5.2 Surfaces and bundles	9
5.3 Rendering approach	11
5.4 Core principles	11
6. Technology Stack	11
7. Angular Project Structure	13
8. Domain/Feature Boundaries	14
9. Routing Architecture	16
9.1 Principles	16
9.2 Route map	16
Public storefront (StorefrontShell)	16
Auth (AuthShell)	19
Customer (AccountShell , guard authGuard('customer'))	19
Admin (AdminShell , canMatch: adminMatch , canActivate: adminGuard)	19
9.3 Redirect rules	20
10. Authentication & Authorization	21
10.1 Token model (CONFIRMED by backend plan)	21
10.2 AuthStore states	21
10.3 Session restoration	21
10.4 Interceptors and refresh strategy	21
10.5 Guest order access	22
10.6 Guards	23
10.7 Auth flows	23
11. API Integration Architecture	24
11.1 Base layer	24
11.2 Query/pagination contract	24
11.3 Frontend API services	24
11.4 Error behavior per call (summary; full strategy in §33)	29
12. State Management	29
12.1 Decision	29
12.2 Where state lives	29
12.3 Store shape (all feature stores)	29
12.4 Effects (side-effect rules)	30
12.5 Anti-patterns to avoid	30
13. Internationalization / RTL / LTR	30
14. Design System Implementation	31
14.1 Token architecture	31
14.2 Global vs feature styles	32
14.3 Primitive inventory (Design System §21 → Angular)	32
15. Shared Components	33
Common page contract (applies to every page spec in §16-§19)	34
16. Storefront Architecture	35
16.1 Shell	35
16.2 Page specifications	35
17. Customer Dashboard Architecture	39
18. Student Services Architecture	43
18.1 Separation rules (CONFIRMED; enforce in code review)	43
18.2 Page specifications	43
18.3 Dynamic form renderer	46
19. Admin Dashboard Architecture	46
19.1 Shell	46
19.2 Admin conventions (apply to every Admin page)	46
19.3 Page specifications	46
20. Notifications & Real-Time Architecture	53
20.1 Principles	53
20.2 RealtimeService	53
20.3 Notification UI	54
20.4 Flow 10 (Backend event → UI)	54
21. Forms & Validation	54

AL-AZHARI LIBRARY	FRONTEND IMPLEMENTATION PLAN
21.1 Strategy	54
21.2 Form inventory	55
22. Search / Filtering / Pagination	56
23. Cart & Checkout	56
23.1 Cart state (CartStore , global)	56
23.2 Checkout flow (CheckoutStore , route-scoped to /checkout/**)	57
23.3 Flows	60
24. Orders & Fulfillment	60
24.1 Status typing	60
24.2 Order detail page (/orders/:reference) — ORD-001	60
24.3 Tracking (/orders/:reference/tracking) — ORD-002	61
24.4 Shipping adjustments (OD-03/04)	61
24.5 FLOW 7 — Admin order (pending → fulfillment)	61
25. Payment Proof Workflow	62
25.1 When it applies (CONFIRMED)	62
25.2 Timing: what exactly happens at checkout (CONTRACT TO VERIFY; FB-04)	62
25.3 Customer flow (FLOW 3)	62
25.4 Upload mechanics (\$29): POST ../upload-config → direct Cloudinary upload → POST ../payment-proofs with returned metadata (+ optional note + idempotency key).	62
25.5 Admin flow (FLOW 8)	62
25.6 States on the customer side	62
26. Pre-order Workflow (FLOW 5)	63
27. Service / Quotation Workflow (FLOW 6)	63
28. Returns / Refunds (FLOW 9)	64
29. File Uploads	64
29.1 PaymentProofUploader	65
29.2 Product images (Admin)	65
30. SEO	66
30.1 Decision FB-02 — SSR / prerender / SPA (OPEN DECISION — DO NOT INVENT)	66
30.2 SeoService (public pages only)	66
31. Accessibility	67
32. Responsive Design	68
33. Error Handling	69
33.1 Pipeline	69
33.2 Status → behavior	69
33.3 Message rules	70
34. Loading / Empty / Error States	70
35. Security	71
36. Performance	72
37. Testing Strategy	73
38. Implementation Phases	74
Phase 0 — Foundation and decisions	74
Phase 1 — Design system and shared primitives	74
Phase 2 — Core infrastructure and API layer	75
Phase 3 — Authentication	75
Phase 4 — Storefront and catalog	75
Phase 5 — Cart and checkout	75
Phase 6 — Orders and customer dashboard	76
Phase 7 — Services and quotations (customer side)	76
Phase 8 — Notifications and Socket.IO	76
Phase 9 — Admin foundation	76
Phase 10 — Admin commerce	76
Phase 11 — Admin payments, services, pre-orders, returns/refunds	77
Phase 12 — Reports, content, coupons, customers, audit, settings	77
Phase 13 — SEO, accessibility, performance	77
Phase 14 — Testing and production hardening	77
39. Definition of Done	77
40. Traceability Matrix	78
41. Frontend Open Decisions / Blockers	82
41.1 Frontend-specific decisions	82
41.2 Inherited source decisions that affect the frontend	86
42. Final Recommended Implementation Sequence	86

FRONTEND IMPLEMENTATION PLAN

Version 1.0 · Angular frontend · Arabic-first / RTL-first

Status	Implementation baseline for Frontend Developer hand-off; PASS WITH OPEN DECISIONS
Prepared	4 October 2026
Consumes	Backend REST API <code>/api/v1</code> (Node.js · Express · TypeScript · MongoDB · Socket.IO)
Source set read in full or in the relevant parts	PRD v1.0, Master Product Brief v2.0, IA & Sitemap, MongoDB Implementation Plan, Backend Implementation Plan, DESIGN-SYSTEM.md, UX-UI-SPECIFICATION.md, DESIGN.md, Website Design Direction PDF
Not accessible when this plan was written	<code>design-website/</code> (code.html, screen.png, per-screen DESIGN.md) and the backend repository, OpenAPI file, and Postman collection. See §4.3.

Labels used in this document

Label	Meaning
CONFIRMED	Stated in a governing source. Preserve.
FRONTEND DECISION	Architecture choice made in this plan. It creates no business rule.
CONTRACT TO VERIFY	The backend plan implies this API behavior but does not fully specify it. Check it against the OpenAPI file or Postman collection before coding. Do not guess.
OPEN DECISION — DO NOT INVENT	The owner has not decided. Keep it configurable or visibly pending.
FRONTEND BLOCKER	Work in a specific area must pause (or use the stated safe behavior) until resolved.

1. Executive Summary

AL-AZHARI LIBRARY is the digital extension of a real library in Qena. The frontend is one Angular application with three isolated surfaces:

1. Storefront: home, catalog, search, product detail, cart, checkout, order confirmation and tracking, pre-orders, services (separate lifecycle), contact.
2. Customer area: auth, profile, addresses, orders, payments, service requests and quotations, pre-orders, returns and refunds, notifications.
3. Admin shell: Owner and Admin operations (orders, payments, catalog, inventory, services, quotations, pre-orders, shipping, returns, refunds, customers, content, coupons, reports, audit, settings).

The plan makes these architectural decisions up front, so the developer does not have to:

- Standalone, lazy-loaded Angular with signals. Feature-level signal stores, RxJS only for streams and HTTP. No NgRx. No global store.
- A typed API layer under `core/api` that maps every backend module to a sensible frontend domain service. DTOs are mapped to domain models at the boundary. Money stays integer minor units (EGP) until the display pipe.
- Backend is the only authority for state, price, stock, permissions, and transitions. The frontend renders `allowedActions`-style information only where the backend provides it. Otherwise it renders the action only for the states documented in the PRD and always handles `409/422` as a normal outcome (see §10.6, §24).
- Services are a separate feature area (`features/services`) with its own models, routes, and state. There is no "service product" abstraction and no shared checkout.
- Socket.IO is a convenience layer behind an interface. Every screen works with plain HTTP refresh; a socket event only triggers revalidation of persisted state.
- Runtime i18n and direction switching inside one build. Arabic/RTL is the default; English/LTR is a second dictionary, not a second app.
- SSR-ready code regardless of the SSR decision (FB-02): no unguarded `window` / `document` use, all meta through one `SeoService`.
- The visual system is the existing Design System. It is implemented as SCSS plus CSS custom properties and a small set of own components. No UI framework.

Everything the sources leave open is catalogued in §41 with a safe interim behavior. The three items that most affect what a developer builds first are the final payment-method list (OD-20), the service-form fields (OD-12), and the styling/SSR/design-file decisions in §41 (FB-01 to FB-03).

2. Frontend Goals

#	Goal	How the plan achieves it
G1	Books-first discovery and purchase on a phone	Mobile-first layouts, URL-driven catalog, product card shows identity/price/availability/next action (§16, §22)
G2	Honest process states	One shared status presentation map for order, payment, service, pre-order, return, refund (§15, §24)
G3	Services are never products	Separate feature, models, routes, state; no cart interaction (§18, §27)
G4	Human support stays one tap away	Contextual WhatsApp component with per-context labels; never a state-changing action (§15, §20)
G5	Guest checkout always works	Guest cart and guest order token handling (§23)
G6	Backend remains authoritative	No client-side totals as truth, no client-only authorization, version-conflict handling (§10, §11, §33)
G7	Arabic-first, accessible, resilient	RTL tokens, CDK a11y, designed loading/empty/error states (§13, §31, §34)
G8	Open decisions stay open	Settings-driven UI and visible "pending" treatment (§41)
G9	Verifiable coverage	Traceability matrix PRD → route → component → API → test (§40)

Non-goals (CONFIRMED exclusions): reviews/ratings, LMS, marketplace, automated payment gateway, map pin, phone OTP, in-site service file upload, a second checkout for services, a permanent season-only catalog, unqualified "best/number one" copy.

3. Source Documents & Authority

Authority order (as given in the brief and used by the backend plan):

1. PRD v1.0 explicit functional decisions
2. Master Product Brief v2.0 business facts
3. IA & Sitemap (routes, domain boundaries)
4. MongoDB Implementation Plan (data-model expectations)
5. UX/UI Specification (UI data contracts, interaction)
6. Design System (visual, RTL, accessibility)
7. Backend Implementation Plan (API contracts)

Design Direction PDF and DESIGN.md are treated as design-intent documents under level 6.

Where sources disagree, this plan follows the higher authority and surfaces the conflict:

Conflict	Handling
Payment methods: PRD lists COD + InstaPay, Vodafone Cash, Orange Cash, Etisalat Cash, WE Pay; Master Brief lists COD, cash at library, InstaPay, Vodafone Cash	OD-20. The payment step renders whatever the backend returns as enabled methods. No list is hard-coded.
Open-decision numbering: the IA/UX/DB documents use OD-19 for accessibility/performance/availability targets; the Backend plan uses OD-19 for operational notification/email policy and adds OD-22 (proof retention), OD-23 (backup RPO/RTO), OD-24 (post-acceptance non-financial order edits)	This plan refers to decisions by name and by its own FB-ID, and cites the source OD number only where it is unambiguous.
Route identifier: IA uses <code>:id</code> for orders and service requests; the backend addresses them by <code>reference</code> (e.g. <code>ORD-20260923-0001</code>)	FRONTEND DECISION: route params are named <code>:reference</code> ; URL path shapes are unchanged from the IA. Private records use references, not Mongo IDs.
Styling tech: the brief for this plan names SCSS; the Design System, UX spec, and DESIGN.md name Tailwind	FB-01.
Payment-proof endpoint path: the endpoint table shows <code>/orders/:reference/payment-proofs</code> ; the traceability table shows <code>/payment-proofs/*</code>	CONTRACT TO VERIFY. This plan uses the endpoint-table form.

4. Existing System Analysis

4.1 Backend (consumed, not rebuilt)

Modular monolith under `/api/v1`. Modules: auth, users, addresses, products, categories, inventory, carts, orders, preorders, payments, shipping, coupons, refunds, returns, services, quotations, admin, audit, content, notifications, reports, settings. Cross-cutting: Zod validation, Argon2id, Pino, Socket.IO, Cloudinary (direct signed upload), email through an outbox worker, Jest tests.

Facts from the backend plan that shape the frontend:

- Envelope: success is `{ success: true, data, meta: { requestId, pagination } }`; error is `{ success: false, error: { code, message, fields?: [{ path, code } ] }, meta: { requestId } }`.

- **Pagination:** `meta.pagination = { limit, nextCursor }`. Cursor pagination is preferred for operational queues; page-number pagination is allowed for the public catalog if documented. Which one the catalog uses is a CONTRACT TO VERIFY.
- **Auth:** short-lived JWT access token in `Authorization: Bearer`; opaque refresh token in an `HttpOnly`, `Secure` cookie; refresh rotates; reuse of a rotated token fails. Claims carry `sub`, `role`, `sessionId`, `tokenVersion`. Same-origin deployment is the default; CORS is strict with credentials only for the refresh cookie.
- **Guest orders:** `POST /orders` returns a raw guest access token once. Access requires reference + token. Guests cannot list orders.
- **Errors:** `VALIDATION_ERROR 400`, `AUTH_REQUIRED 401`, `AUTH_INVALID_CREDENTIALS 401`, `AUTH_TOKEN_INVALID (verify/reset) 401`, `FORBIDDEN 403`, `NOT_FOUND 404`, `RESOURCE_CONFLICT 409`, `ORDER_STATE_CONFLICT 409`, `INSUFFICIENT_STOCK 409`, `IDEMPOTENCY_KEY_REUSED 409`, `PAYMENT_PROOF_REQUIRED 422`, `SHIPPING_CONFIGURATION_UNAVAILABLE 422`, `BUSINESS_RULE_VIOLATION 422`, `RATE_LIMITED 429`, `INTERNAL_ERROR 500`, `DEPENDENCY_UNAVAILABLE 503`; checkout also returns `PRICE_CHANGED`, `AVAILABILITY_CHANGED`, `PAYMENT_METHOD_UNAVAILABLE`.
- **Optimistic concurrency:** mutations take `expectedVersion / version`; a stale write returns 409.
- **Money/dates:** integer minor units (`amountMinor`, `currency: "EGP"`); UTC ISO dates; the client formats in `Africa/Cairo`.
- **Localized content:** `{ ar: string, en?: string }`. English may be absent.
- **Media:** product images via Cloudinary keys/URLs; payment proofs are private and only viewable through short-lived signed URLs, on demand, for authorized roles.

4.2 Endpoints the backend plan specifies (\$48 of the backend plan)

Auth: `POST /auth/register|login|refresh|logout|verify-email|forgot-password|reset-password`, `GET /me`.
Catalog: `GET /categories`, `GET /products`, `GET /products/:slug`, `GET /search`, `GET /content/home`.
Cart/checkout: `GET /cart`, `POST /cart/items`, `PATCH|DELETE /cart/items/:itemId`, `POST /checkout/shipping-estimate`, `POST /orders`, `GET|PATCH /orders/:reference`, `POST /orders/:reference/cancel`. **Payments:** `GET /orders/:reference/payment`, `POST /orders/:reference/payment-proof/upload-config`, `POST /orders/:reference/payment-proofs`, `POST /admin/payments/:paymentId/confirm|reject|request-new-proof`. **Admin orders/inventory/shipping:** `GET /admin/orders`, `POST /admin/orders/:reference/accept|reject|status|shipping|cod-confirmation-reminder`, `POST /admin/inventory/:productId/adjust`. **Services:** `GET /services`, `GET /services/:slug`, `POST /services/:slug/requests`, `GET /service-requests/:reference`, `POST /admin/service-requests/:reference/quotes`, `POST /service-requests/:reference/quotation/accept|reject`. **Pre-orders/returns/refunds:** `POST /products/:slug/pre-orders`, `GET /pre-orders`, `POST /admin/pre-orders/:reference/accept`, `POST /orders/:reference/returns`, `POST /admin/returns/:reference/approve|reject`, `POST /admin/refunds/:refundId/complete`. **Admin catalog/settings/reports:** `POST /admin/products`, `PATCH /admin/products/:id`, `POST /admin/categories`, `PATCH /admin/settings/:key`, `GET /admin/reports/:report`, `GET /admin/audit-logs`; `/addresses` is named in the traceability table.

4.3 Gaps found, and why they matter (read before starting)

Gap A — Design files not accessible. `design-website/` (code.html, screen.png, DESIGN.md per screen) was not available to the author of this plan. The brief requires an exact reproduction of those screens. Therefore:

- Page specs below derive layout, states and content from IA, UX/UI Specification, DESIGN-SYSTEM, DESIGN.md, and the Design Direction PDF.
- Every page spec lists a Design source column pointing to the expected `design-website/<screen>` item. Phase 1, task 1 is a screen audit: open each mockup, record layout/spacing/states against the page spec, and amend the spec where the mockup is more specific. If the mockup contradicts a CONFIRMED business rule, the business rule wins and the conflict is logged.

- This does not weaken any architectural decision in this plan.

Gap B — The backend endpoint table is not exhaustive. The following are needed by the documented screens but are not listed in the backend plan's endpoint tables. Treat every one as **CONTRACT TO VERIFY** (check the OpenAPI/Postman collection). Do not invent paths or fields. If a needed endpoint does not exist, raise it as a backend ticket and keep the screen on its documented empty/pending state.

Needed by	Missing from the endpoint tables
Account	profile update (<code>PATCH /me ?</code>), addresses CRUD (only <code>/addresses</code> is named), "my orders" list, "my payments" list, "my service requests" list, notification list / mark-read / unread count, return detail, refund view
Checkout	enabled payment methods + payment details (backend says they come from <code>settings.payment.methods</code> ; the public read path is unspecified), contact/WhatsApp link data, address/governorate list for the form
Guest orders	exact way the guest token is sent (header name or link parameter) and how the guest session identifier for carts is carried
Cart	whether guest→user cart merge happens implicitly on login or by an explicit call; the structured shape of the merge conflict list
Pre-orders	customer accept/pay action after Admin acceptance; pre-order detail
Orders	COD customer-confirmation action endpoint
Admin	lists/detail for products, categories, customers, services (catalog and request queue), service catalog edit, pre-orders, returns, refunds, shipping queue, coupons, content modules, notifications, payments queue (<code>GET /admin/payments</code>), signed proof-URL retrieval, settings read, role/permission read
Realtime	whether room joins are automatic on handshake or via client <code>join</code> events
Catalog	sort options accepted by <code>/products</code> and <code>/search</code> ; response shape for "active filters" and counts
Public config	<code>GET</code> for shipping serviceability copy, business contact, business hours

Gap C — Source-level decisions are open (§41): payment list, service form fields, shipping rules, return/refund policy, coupon governance, verification gating, SSR/prerender, styling engine, English URL strategy, and others.

Gap D — No existing Angular code. Everything is greenfield; there is no legacy to preserve.

5. Frontend Architecture

5.1 Layered model

features/*	(page components, feature components, feature state services, feature-specific forms/mappers)
layout/*	(StorefrontShell, AccountShell, AdminShell, auth)
shared/*	(ui primitives, form controls, data-display, pipes, directives, status presentation)
domain/*	(domain models, enums, state maps, mappers)
core/*	(api, auth, http, realtime, i18n, seo, config, error handling, storage, analytics, ally)

▲ dependency direction: upper layers may import lower layers only

Import rules (enforce with ESLint boundaries):

1. `core` imports nothing from `features`, `layout`, or `shared`.
2. `domain` imports only from `core/types`-level primitives (no Angular).
3. `shared` imports `core` and `domain`; never `features`.
4. `features/<a>` never imports from `features/<b>`. Cross-feature needs go through `domain` or `core`.
Exception: `features/storefront` may import the public `PriceDisplay`-style components from `shared`, not from other features.
5. `layout` imports `core`, `shared`; features are mounted by the router, never imported by layouts.
6. `features/admin` is a separate lazy bundle and must not be imported by storefront or customer code.

5.2 Surfaces and bundles

Surface	Entry	Shell	Lazy bundle	Auth
Public storefront	/	StorefrontShell	features/storefront , features/checkout , features/services (public part)	Guest or customer
Auth	/login ...	AuthShell (minimal)	features/auth	Guest only (except verify/reset)
Customer	/account/**	AccountShell inside StorefrontShell	features/customer	Customer
Admin	/admin/**	AdminShell	features/admin (further split per module)	Admin or Owner

5.3 Rendering approach

FRONTEND DECISION: build everything SSR-safe (no direct `window`, `document`, `localStorage`, `navigator` access outside `PlatformService` and `isPlatformBrowser` guards). Whether SSR/prerender is actually deployed is FB-02. Browser-only things (Socket.IO, file inputs, `matchMedia`) are created only in the browser.

5.4 Core principles

- Backend state is truth. Client state is a cache with a version.
- One owner per piece of state (see §12).
- All HTTP goes through `core/api` services. Components never call `HttpClient`.
- All user-visible strings come from translation keys (§13). All status text comes from the shared status map.
- All money passes through `MoneyPipe` / `MoneyService`. Never use floating-point math for money in the UI.
- All dates through `DatePipe`-style locale pipe set to `Africa/Cairo`.
- Components are `ChangeDetectionStrategy.OnPush`, standalone, signal-based inputs/outputs where available in the chosen Angular version.
- No component over about 300 lines of template+class logic. Split by responsibility.

6. Technology Stack

Concern	Choice	Rationale
Framework	Angular (current stable at kickoff; record the exact major in <code>package.json</code> and <code>docs/</code>) with standalone components, new control flow (<code>@if/@for/@switch</code>), signals, <code>inject()</code>	Brief mandates Angular + TS. Exact version is not specified in the sources: pin it at kickoff.
Language	TypeScript <code>strict</code> , <code>noUncheckedIndexedAccess</code> , <code>exactOptionalPropertyTypes</code>	Typed DTO/domain separation
Styling	SCSS + CSS custom properties (design tokens) + logical properties	FB-01 records the Tailwind question
Behavior/ally	<code>@angular/cdk</code> (overlay, dialog, ally, layout/BreakpointObserver, portal, scrolling, clipboard, table where useful)	Brief
State	Angular signals + RxJS; feature-level injectable stores	§12
HTTP	<code>HttpClient</code> with functional interceptors	§11
Forms	Typed reactive forms	§21
i18n	One runtime i18n library (recommended: Transloco, lazy scoped dictionaries, runtime locale switch).	Angular's built-in compile-time <code>@angular/localize</code> produces one build per locale, which contradicts "do not build two apps" and runtime switching. A library here is justified. A thin in-house translate pipe is an acceptable alternative if the team prefers zero dependencies; the abstraction in §13 must be the same either way.
Realtime	<code>socket.io-client</code>	Matches the backend; behind an <code>RealtimeService</code> interface

Concern	Choice	Rationale
Icons	Lucide through an Angular-compatible package (Design System §15), bundled as an SVG icon registry, only the used icons	Design System
Fonts	IBM Plex Sans Arabic + Inter, self-hosted (<code>font-display: swap</code>), subset to Arabic + Latin	Design System §8; confirm self-hosting vs Google Fonts at FB-01 time (same-origin CSP)
Testing	Jasmine/Karma or Jest (team choice; FRONTEND DECISION: Jest + Angular testing utilities is acceptable), Testing Library optional, Playwright for E2E	§37
Lint/format	ESLint (+ Angular ESLint, boundaries plugin), Prettier, Stylelint for SCSS (logical-property rule)	Enforce RTL discipline
Not used	NgRx, Angular Material visual components, PrimeNG, Bootstrap, a date library (use <code>Intl</code>), a money library, Lodash, any carousel/modal lib that duplicates CDK	Avoid unnecessary libraries

Config: `environment.*.ts` holds only non-secret values: `apiBaseUrl` (`/api/v1` , relative for same-origin), `socketUrl` (origin), `cloudinaryCloudName` only if the backend's signed config does not return it, `defaultLocale: 'ar'` , feature flags for open decisions. No secrets or API keys of any kind. Cloudinary upload URL and signature are returned by `upload-config` at runtime.

7. Angular Project Structure

```

src/
├─ main.ts / main.server.ts (if FB-02 = SSR)
├─ index.html                      lang="ar" dir="rtl" default; preconnect fonts
├─ app/
│   ├─ app.config.ts               providers: router, http interceptors, i18n, APP_INITIALIZER
│   ├─ app.routes.ts               top-level lazy route groups only
│   └─ core/
│       ├─ api/                   ← all HttpClient usage lives here
│       │   ├─ base/               ApiClient, envelope types, pagination types, query builders
│       │   ├─ dto/                 API DTO interfaces, grouped by backend module
│       │   ├─ mappers/             DTO ↔ domain mappers (pure functions)
│       │   ├─ public/              CatalogApi, ContentApi, SearchApi, ConfigApi
│       │   ├─ commerce/            CartApi, CheckoutApi, OrdersApi, PaymentsApi, PreOrdersApi, ReturnsApi
│       │   ├─ services/            ServiceCatalogApi, ServiceRequestsApi
│       │   ├─ account/             AuthApi, MeApi, AddressesApi, NotificationsApi
│       │   └─ admin/               Admin*Api services (one per admin module)
│       ├─ auth/                   AuthService, AuthStore, token holder, guards, session bootstrap
│       ├─ http/                   interceptors (auth, error, locale, request-id, idempotency)
│       ├─ errors/                 ApiError, error-code map, ErrorMapperService, GlobalErrorHandler
│       ├─ realtime/               RealtimeService (socket lifecycle), event router, dedupe
│       ├─ i18n/                   LocaleService, DirectionService, translation loader, formatters
│       ├─ seo/                    SeoService, route meta resolver, JSON-LD builders (public only)
│       ├─ config/                 AppConfigService (public settings), feature flags, runtime config
│       ├─ storage/                 PlatformService, SafeStorage (SSR-safe), guest-token store
│       ├─ analytics/              AnalyticsService (event facade, no-op default)
│       ├─ ally/                   LiveAnnouncer wrapper, focus restore helpers
│       └─ util/                   idempotency key, cursor helpers, query-param serializers
│   └─ domain/
│       ├─ money/                  Money type, formatting rules
│       ├─ catalog/                Product, Variant, Category, Availability, filters model
│       ├─ cart/                   Cart, CartLine, CartChangeSet
│       ├─ orders/                 Order, OrderStatus, OrderTimeline, PaymentStatus, Fulfillment
│       ├─ services/               ServiceCategory, ServiceRequest, Quotation, ServiceStatus
│       ├─ preorders/ returns/ refunds/ notifications/ account/ admin/
│       └─ status/                 StatusPresentation map (label key, tone, icon) shared by all surfaces
│   └─ shared/
│       ├─ ui/                     Button, IconButton, Link, Badge, StatusBadge, Alert, Card, Surface,
│       │                           Tabs, Breadcrumbs, Pagination/LoadMore, Skeleton, EmptyState,
│       │                           ErrorState, Spinner, Tooltip, Divider, Avatar
│       ├─ overlay/                Dialog (ConfirmDialog), Drawer, BottomSheet, Toast, Menu (CDK based)
│       ├─ forms/                  FormField, TextInput, PhoneInput, EmailInput, PasswordInput, Textarea,
│       │                           Select, Radio/RadioGroup, Checkbox, Switch, QuantityStepper,
│       │                           FileDropzone, ErrorSummary, server-error mapper, validators
│       ├─ data/                   DataTable (+ responsive card fallback), Timeline, KeyValueList,
│       │                           PriceDisplay, AvailabilityBadge, MoneyPipe, DatePipe, ReferenceText
│       ├─ commerce/               ProductCard, ProductGrid, CartLine, OrderSummary, CouponField,
│       │                           ShippingSummary, EstimateVsFinalCost
│       └─ support/                 WhatsAppCta (context-aware), ContactBlock
│   └─ layout/
│       ├─ storefront-shell/        StoreHeader, StoreFooter, MobileNav, SearchBar, CartMini, WhatsApp float
│       ├─ account-shell/           AccountNav (sidebar desktop / stacked mobile)
│       ├─ admin-shell/             AdminSidebar, AdminTopbar, AdminDrawerNav, RoleBadge, NotificationBell
│       └─ auth-shell/
│   └─ features/
│       └─ storefront/              home, shop, catalog (all, category, search), product-detail, contact, not-
found
│   └─ checkout/                   cart, checkout steps, order confirmation (guest/customer)
│   └─ orders/                     order detail, tracking, return request (reachable by guest or customer)
│   └─ preorders/                  pre-order request page (public)
│   └─ services/                   landing, catalog, detail, request (public part)
│   └─ auth/                       login, register, verify-email, forgot/reset
│   └─ customer/                   account overview, profile, orders list, addresses, payments,
│                                   service requests + quotation, pre-orders, returns, notifications

```

└─ admin/	dashboard, orders, payments, products, categories, inventory, customers, services, preorders, shipping, returns, refunds, notifications, content, coupons, reports, audit, settings
└─ routing/	route data typings, guards registry, title/breadcrumb strategy, preloading strategy, redirect rules
└─ assets/ (icons sprite, i18n/ar.json en.json split by scope, placeholders)	
└─ styles/	
└─ tokens/	_color.scss _type.scss _space.scss _radius.scss _shadow.scss _motion.scss _z.scss _breakpoints.scss _containers.scss
└─ base/	reset, typography, direction helpers (:dir()), focus, utilities
└─ themes/	light.scss (only theme in V1)
└─ styles.scss	imports in order: tokens → base → themes

Where each thing lives

Thing	Location	Reason
Anything that calls the network	core/api	One place to change contracts
Types shaped like the backend	core/api/dto	They die at the mapper
Types the UI reasons about	domain/*	Stable even if the API changes
A component used by two or more features	shared/*	Reuse before invent
A component used by one feature	that feature's components/	No premature sharing
Feature state	features/<f>/state/*.store.ts	Scoped to the route subtree via providers
Feature forms	features/<f>/forms/*	Typed form factories and validators

8. Domain/Feature Boundaries

Domain	Frontend feature	API service(s)	Domain models	State owner	Hard boundary
Catalog & content	storefront	CatalogApi , SearchApi , ContentApi	Product, Variant, Category, ProductFilters, ContentModule	Route-scoped CatalogStore (query state in URL)	Seasonal modules never change category structure
Cart	checkout (cart page) + global CartStore	CartApi	Cart, CartLine, CartChangeSet	Global CartStore (header badge, product pages)	Fixed-price products only; services rejected
Checkout & orders	checkout , orders	CheckoutApi , OrdersApi , PaymentsApi	Order, OrderItem, Fulfillment, Totals, Payment, PaymentProof	CheckoutStore (route-scoped, draft only); OrderDetailStore	Totals are rendered from server responses, never summed client-side as truth
Pre-orders	preorders + customer list	PreOrdersApi	PreOrder	PreOrdersStore (customer)	No second catalog; no stock reservation shown
Services	services	ServiceCatalogApi , ServiceRequestsApi	ServiceCategory, ServiceFormField, ServiceRequest, Quotation	ServiceRequestStore	No cart, no product checkout, no attachment control
Returns/refunds	orders (request) + customer (detail)	ReturnsApi	ReturnRequest, Refund	ReturnStore	Refund is a manual Admin record; no refund action for customers
Auth/account	auth , customer	AuthApi , MeApi , AddressesApi	User, Session, Address	Global AuthStore	Roles are display hints only
Notifications	customer + admin bell	NotificationsApi , RealtimeService	Notification	Global NotificationStore	Persisted list is truth; socket events trigger refresh
Admin	admin/*	Admin*Api	Admin views of the above	One store per admin module, route-scoped	Server decides permissions
Platform	core	ConfigApi	AppConfig (public settings), FeatureFlags	Global AppConfigStore	Open decisions are config, not code

Customer-vs-guest access to orders: `features/orders` is mounted under the storefront shell and serves both guests (with a token) and customers (with session), because the IA marks `/orders/:reference*` as "conditional" access. The `OrderAccessResolver` (§10.5) picks credentials.

9. Routing Architecture

9.1 Principles

- Top-level `app.routes.ts` contains only lazy group roots.
- Paths follow the IA exactly. No invented routes.
- Guards are functional (`CanActivateFn` / `CanMatchFn`). `CanMatch` is used on the admin group so the admin bundle is not even downloaded by non-admins.
- Titles and SEO meta come from route `data` plus a `resolve` (for product/category/service). Private areas get `noindex`.
- Preloading: `NetworkAwarePreloading` — preload storefront-critical routes (catalog, product, cart) after first paint on good connections; never preload `/admin` for customers.
- Scroll restoration on; focus moves to the page `<h1>` on route change (a11y, §31).
- Unknown paths → `/404` (`NotFoundPage`); denied authenticated → `/403`.

9.2 Route map

Access legend: G guest, C customer, A admin/owner, *G(token)** guest with order token, Any.

Public storefront (`StorefrontShell`)

Path	Page component	Access / guards	Resolver / preload	Query params	SEO
/	HomePage	Any	ContentApi.home() (non-blocking, skeleton)	—	index; home meta
/shop	ShopLandingPage	Any	categories	—	index
/shop/all	CatalogPage	Any	—	q? category? educationType? stage? grade? subject? publisher? priceMin? priceMax? availability? sort? cursor/page	index (canonical strips volatile params)
/categories	CategoryIndexPage	Any	categories	—	index
/categories/:slug	CatalogPage (category mode)	Any	category by slug (404 on miss)	same filters	index
/search	CatalogPage (search mode)	Any	—	q + filters	noindex on result pages (FRONTEND DECISION: avoid thin-content indexing); confirm with owner (FB-14)
/products/:slug	ProductDetailPage	Any	product by slug (404 on miss)	variant?	index; product meta + availability
/products/:slug/pre-order	PreOrderRequestPage	Any (see §26)	product by slug; guards eligibility	variant?	noindex
/cart	CartPage	Any	cart	—	noindex
/checkout	redirect → /checkout/customer	Any	requires non-empty cart (redirect /cart)	—	noindex
/checkout/customer	CheckoutCustomerPage	Any	—	—	noindex
/checkout/fulfillment	CheckoutFulfillmentPage	Any	requires step 1 valid	—	noindex
/checkout/payment	CheckoutPaymentPage	Any	enabled payment methods	—	noindex
/checkout/payment-proof	CheckoutPaymentProofPage	Any	only if selected method needs proof (see §25)	—	noindex
/checkout/review	CheckoutReviewPage	Any	revalidation call	—	noindex

Path	Page component	Access / guards	Resolver / preload	Query params	SEO
/orders/:reference/confirmation	OrderConfirmationPage	G*(token) or C	order access resolver	t? (guest token link, stripped immediately — §10.5)	noindex
/orders/:reference	OrderDetailPage	G*(token) or C owner	order access resolver	—	noindex
/orders/:reference/tracking	OrderTrackingPage	same	same	—	noindex
/orders/:reference/return	ReturnRequestPage	C owner (guest: see FB-17)	order + eligibility	—	noindex
/services	ServicesLandingPage	Any	service categories	—	index
/services/catalog	ServiceCatalogPage	Any	service categories	—	index
/services/:slug	ServiceDetailPage	Any	service by slug	—	index; no price in meta
/services/:slug/request	ServiceRequestPage	Any (guest allowed; IA marks auth "conditional")	service form config	—	noindex
/contact	ContactPage	Any	public config	—	index
/403 , /404	ForbiddenPage , NotFoundPage	Any	—	—	noindex

Service request for guests. The IA marks `/services/:slug/request` as auth "Conditional" and the backend accepts Guest/Customer on `POST /services/:slug/requests`; but the request detail and quotation routes are customer-only (`/account/services/:reference`). A guest therefore gets no way to see the quotation. How a guest follows or accepts a quotation is FB-09 (OPEN DECISION — DO NOT INVENT). Safe interim: the request page shows a clear "sign in to track your request" path and, for a signed-out user, offers login/register before submit while still letting the backend decide. Do not build a guest quotation-view route.

Auth (`AuthShell`)

Path	Page	Access / guard	Notes
<code>/login</code>	<code>LoginPage</code>	<code>guestOnly</code> (redirect authenticated to <code>returnUrl</code> or <code>/account</code> ; admins to <code>/admin</code>)	<code>returnUrl</code> query param, validated as an internal path
<code>/register</code>	<code>RegisterPage</code>	<code>guestOnly</code>	
<code>/verify-email</code>	<code>VerifyEmailPage</code>	Any (token in query <code>token</code>)	Consumes token on load via POST; strips param from URL
<code>/forgot-password</code>	<code>ForgotPasswordPage</code>	<code>guestOnly</code>	Neutral success message always
<code>/reset-password</code>	<code>ResetPasswordPage</code>	Any (token in query)	Strips token from URL after reading

Customer (`AccountShell`, guard `authGuard('customer')`)

Path	Page	Notes
<code>/account</code>	<code>AccountOverviewPage</code>	current orders, services, notifications
<code>/account/profile</code>	<code>ProfilePage</code>	
<code>/account/orders</code>	<code>AccountOrdersPage</code>	links to <code>/orders/:reference</code>
<code>/account/addresses</code>	<code>AddressesPage</code>	
<code>/account/payments</code>	<code>PaymentHistoryPage</code>	
<code>/account/services</code>	<code>AccountServicesPage</code>	
<code>/account/services/:reference</code>	<code>ServiceRequestDetailPage</code>	
<code>/account/services/:reference/quotation</code>	<code>QuotationReviewPage</code>	
<code>/account/pre-orders</code>	<code>AccountPreOrdersPage</code>	
<code>/account/pre-orders/:reference</code>	<code>PreOrderDetailPage</code>	
<code>/account/notifications</code>	<code>NotificationsPage</code>	
<code>/account/returns/:reference</code>	<code>ReturnDetailPage</code>	return state + refund record

Admin and Owner users who open `/account/**` are not blocked by the frontend (the backend decides what `/me` returns), but the shell shows a link to `/admin`.

Admin (`AdminShell`, `canMatch: adminMatch`, `canActivate: adminGuard`)

Path	Page	Required permission (backend naming from the plan)
<code>/admin</code>	<code>AdminDashboardPage</code>	any admin role
<code>/admin/orders</code> , <code>/admin/orders/:reference</code>	<code>AdminOrdersQueuePage</code> , <code>AdminOrderDetailPage</code>	<code>orders.read</code> (+ <code>orders.accept</code> , <code>orders.reject</code> , <code>orders.write</code>)

Path	Page	Required permission (backend naming from the plan)
<code>/admin/products</code> , <code>/admin/products/new</code> , <code>/admin/products/:id/edit</code>	list, editor	<code>products.write</code> (read permission name: CONTRACT TO VERIFY)
<code>/admin/categories</code>	<code>AdminCategoriesPage</code>	<code>categories.write</code>
<code>/admin/customers</code> (+ <code>/:id</code>)	list, detail	CONTRACT TO VERIFY
<code>/admin/payments</code>	<code>AdminPaymentsQueuePage</code>	<code>payments.review</code>
<code>/admin/services</code> , <code>/admin/services/requests/:reference</code>	catalog + queue, request detail	<code>services.quote</code> (+ catalog edit permission: CONTRACT TO VERIFY)
<code>/admin/pre-orders</code>	<code>AdminPreOrdersPage</code>	<code>preorders.write</code>
<code>/admin/inventory</code>	<code>AdminInventoryPage</code>	<code>inventory.write</code>
<code>/admin/shipping</code>	<code>AdminShippingPage</code>	<code>shipping.write</code>
<code>/admin/returns</code> , <code>/admin/refunds/:refundId</code>	queue, refund record	<code>returns.write</code> , <code>refunds.write</code>
<code>/admin/notifications</code>	<code>AdminNotificationsPage</code>	any admin role
<code>/admin/content</code>	<code>AdminContentPage</code>	CONTRACT TO VERIFY
<code>/admin/coupons</code>	<code>AdminCouponsPage</code>	CONTRACT TO VERIFY
<code>/admin/reports</code> (+ <code>/:report</code>)	<code>AdminReportsPage</code>	<code>reports.read</code>
<code>/admin/audit</code>	<code>AdminAuditPage</code>	<code>audit.read</code> (backend endpoint exists; the IA sitemap has no audit route and the requested admin scope includes Audit Logs. FRONTEND DECISION: add <code>/admin/audit</code> under Settings navigation; it is justified by the endpoint and the brief)
<code>/admin/settings</code>	<code>AdminSettingsPage</code>	<code>settings.write</code> or Owner

The permission strings above are the backend's candidate keys; the frontend gets the real list for the current user from `GET /me` (CONTRACT TO VERIFY whether permissions are returned). If only `role` is returned, the frontend uses role-level gating for navigation, and the backend's `403` is the real gate (§10.4).

9.3 Redirect rules

- Unauthenticated access to a customer or admin route → `/login?returnUrl=<path>` .
- Authenticated but forbidden → `/403` .
- Authenticated admin signing in from `/login` with no `returnUrl` → `/admin` ; customer → `/account` .
- Checkout steps out of order → the earliest incomplete step.
- Old/unknown product slug → `/404` with a Shop/Search link (no redirect invention).

10. Authentication & Authorization

10.1 Token model (CONFIRMED by backend plan)

Item	Where it lives	Rule
Access token (JWT)	In memory only (<code>AuthTokenHolder</code> signal). Never <code>localStorage</code> / <code>sessionStorage</code> / cookies written by JS.	Sent as <code>Authorization: Bearer</code> .
Refresh token	HttpOnly Secure cookie set by the backend	JS cannot read it. Calls to <code>/auth/refresh</code> and <code>/auth/logout</code> use <code>withCredentials: true</code> .
Session summary	<code>AuthStore</code> signals: <code>status</code> , <code>user</code> , <code>role</code> , <code>permissions?</code>	Derived from login/refresh response and <code>GET /me</code> .

Same-origin deployment is the backend default, so the cookie flows without extra CORS configuration. If the frontend is hosted on a different origin, that is a deployment decision with a CSRF implication that the backend plan itself flags (§50); raise it before doing it.

10.2 AuthStore states

`unknown` (app booting) → `anonymous` | `authenticated` | `refreshing` | `expired` (refresh failed during use) | `forbidden` (page level only).

10.3 Session restoration

`APP_INITIALIZER` runs `AuthService.restore()` :

1. Call `POST /auth/refresh` once (cookie only). If it returns 200: store the access token, call `GET /me` , set `authenticated` .
2. If 401: set `anonymous` silently (no toast, no redirect).
3. If network failure/5xx/timeout: set `anonymous` for rendering, record `restoreFailed=true` , and retry once on the next navigation. Do not block the app boot for more than a short bounded timeout (FRONTEND DECISION: a few seconds; make it a config value).
4. After restore, `CartStore.hydrate()` and `NotificationStore.hydrate()` run (customer only) and the realtime connection starts (§20).

On the server (SSR), `restore()` is skipped; the shell renders as anonymous and the browser restores after hydration. Private pages are `noindex` and not server-rendered with user data.

10.4 Interceptors and refresh strategy

Order of functional interceptors: `requestIdInterceptor` → `localeInterceptor` → `authInterceptor` → `idempotencyInterceptor` → `errorInterceptor` .

`authInterceptor`

- Attaches the bearer token to requests going to `apiBaseUrl` only (never to Cloudinary or any third-party URL).
- On `401 AUTH_REQUIRED` for an authenticated request: run a single shared refresh (`refresh$` with `shareReplay` /in-flight promise). All 401s that arrive while a refresh is in flight wait for it, then retry once.
- A request is never retried twice. A retried request carries a `X-Retry` marker (internal, stripped before sending) so a second 401 goes straight to session-expired handling.

- `/auth/refresh`, `/auth/login`, `/auth/register`, `/auth/logout`, `/auth/verify-email`, `/auth/forgot-password`, `/auth/reset-password` are excluded from refresh-and-retry. A 401 on `/auth/refresh` itself never triggers another refresh. This is what prevents infinite loops.
- If refresh fails: clear token, set `expired`, show one "session ended" notice (single announcement, even if many requests fail), redirect to `/login?returnUrl=...` only when the user is on a protected route; on public pages simply continue as anonymous.
- Do not refresh proactively on a timer based on decoding JWT claims (FRONTEND DECISION: avoid reading token claims for logic). Reactive refresh on 401 is sufficient and avoids clock-skew bugs.

`errorInterceptor` normalizes every failure into `ApiError` (§33). It never swallows errors.

`localeInterceptor` adds `Accept-Language` from the active locale. (CONTRACT TO VERIFY: the backend plan's catalog endpoints accept a `locale` query parameter; use the documented mechanism, header or query, not both.)

`idempotencyInterceptor` adds the idempotency value for calls marked by an `HttpContext` token (checkout, proof submission, refund completion, admin state transitions). The transport form (header `Idempotency-Key` per the MongoDB plan vs body field per the backend's `POST /orders` table) is a CONTRACT TO VERIFY. The key itself is generated by the caller (`IdempotencyKeyService`), persisted for the duration of one logical attempt (kept across retries and reloads of the same attempt), and discarded on success or when the user changes the payload. `409 IDEMPOTENCY_KEY_REUSED` means the payload changed under the same key: generate a new key, show a "please review and submit again" message.

10.5 Guest order access

Aspect	Rule
Token source	The raw guest token is returned once in the <code>POST /orders</code> response.
Storage	FRONTEND DECISION: keep in memory and in <code>sessionStorage</code> , keyed by order reference, through <code>GuestOrderTokenStore</code> . Do not log it, put it in analytics, WhatsApp text, or <code>Referrer</code> -leaking URLs. Cross-session persistence (<code>localStorage</code>) trades convenience against XSS exposure and is FB-16 (OPEN DECISION); the safe default is <code>sessionStorage</code> plus the "save your order link" affordance on the confirmation page.
Secure link	If the backend delivers the token in an emailed link (<code>?t=</code>), the page reads it, stores it per above, and immediately <code>history.replaceState</code> s the URL to remove it. Set <code>Referrer-Policy: same-origin</code> (or stricter) on those routes.
Transport	How the token is sent (header name vs query) is a CONTRACT TO VERIFY. <code>OrderAccessResolver</code> is the only code that knows; everything else asks it for the request options.
Credentials precedence	Authenticated customer → session. Else stored guest token for this reference. Else show the "enter your order link" empty/unauthorized state. A mismatch shows the same neutral not-found/unauthorized state the backend returns.
Guest listing	Guests cannot list orders; no UI offers it.
Merge to account	The backend allows merging a guest order into a verified account with guest proof (§18.4 of the backend plan). Whether and where the UI exposes this is FB-17; do not build a merge UI until the endpoint and policy are confirmed.

10.6 Guards

Guard	Type	Behavior
<code>authGuard(role)</code>	<code>CanActivateFn</code>	Waits for <code>AuthStore.status !== 'unknown'</code> ; <code>anonymous</code> → login with <code>returnUrl</code> ; wrong role → <code>/403</code> .
<code>guestOnlyGuard</code>	<code>CanActivateFn</code>	Authenticated users are redirected away from login/register/forgot.
<code>adminMatch</code>	<code>CanMatchFn</code>	Prevents the admin bundle loading unless role is <code>admin</code> or <code>owner</code> . Non-admin users receive <code>/403</code> if authenticated, login if anonymous.
<code>permissionGuard(key)</code>	<code>CanActivateFn</code>	Hides/blocks a module when permissions are present in <code>/me</code> ; if only role is known, allows the route and relies on backend 403.
<code>ownerGuard</code>	<code>CanActivateFn</code>	Not used in V1. The PRD does not enumerate Owner-only actions. Do not create an Owner-only screen (CONFIRMED in IA/UX). Settings may show actions the backend allows to the user's role.
<code>checkoutStepGuard</code>	<code>CanActivateFn</code>	Enforces step order and non-empty cart.
<code>preOrderEligibleGuard</code>	<code>CanActivateFn</code>	Loads the product; if not out-of-stock + pre-order-eligible, redirect to the product page with a notice.
<code>unsavedChangesGuard</code>	<code>CanDeactivateFn</code>	Admin editors and long forms.

Frontend authorization is presentation only. Hiding a button never replaces the backend check. Every Admin action handles `403 FORBIDDEN` by showing the "no permission" state and leaving the record untouched.

10.7 Auth flows

- Register: `POST /auth/register` (name, phone, email, password). Success → "verify your email" page state (account exists; verification pending). Whether login is allowed before verification and which actions are gated is FB-05 (OPEN DECISION: exact verification gating). Safe behavior: the UI never blocks an action by itself; it surfaces the backend's refusal (`FORBIDDEN` / `BUSINESS_RULE_VIOLATION`) with a "verify your email" prompt and shows an unverified-email banner on the account overview when `/me` indicates it (CONTRACT TO VERIFY field name).
- Login: one field for email or phone plus password. The frontend does not guess which; it sends the identifier as the backend contract requires (CONTRACT TO VERIFY: separate `email` / `phone` fields vs a combined `identifier`). `AUTH_INVALID_CREDENTIALS` → a single generic message; `429` → "try again shortly".
- Logout: `POST /auth/logout` (current-session scope by default; "sign out everywhere" is optional if the contract supports it). Always clear local state even if the call fails; clear cart cache, notification store, socket, guest tokens for this user, then navigate to `/` .
- Verify email: page POSTs the token once; success/invalid/expired states (`AUTH_TOKEN_INVALID`). A "send a new link" action is shown only if the backend provides a resend endpoint (not specified: CONTRACT TO VERIFY).
- Forgot/reset: neutral `202` message always; reset invalidates sessions server-side, so after reset the UI goes to login.
- Session expiration mid-form: forms preserve values; after re-login (`returnUrl`) the user returns to the page, but drafts are re-entered only if held in a route-scoped draft store (checkout draft is kept in memory + sessionStorage without secrets).

11. API Integration Architecture

11.1 Base layer

- `ApiClient` wraps `HttpClient`: prefixes `apiBaseUrl`, unwraps the envelope, returns `Observable<ApiResult<T>>` where `ApiResult` has `data`, `requestId`, and `pagination?`. Errors are thrown as `ApiError { code, httpStatus, message(safe), fields?, requestId, retryable }`.
- DTO → domain mapping in `core/api/mappers`, pure and unit-tested. Components never see DTOs.
- Types: `Money { amountMinor: number; currency: 'EGP' }`; `IsoDateString` branded type; `Localized { ar: string; en?: string }`; typed string-union enums for every status list in the MongoDB/backend plans (§24 lists them). An unknown status string from the API maps to a `unknown` presentation (neutral badge + raw-safe label) and is logged, never thrown.
- Request IDs: log `requestId` from every error; show it in the "technical details" disclosure of the error state so support can find the log.
- Timeouts: default client timeout is configurable (FRONTEND DECISION: a generous default for uploads, shorter for reads). Timeouts become `ApiError('TIMEOUT')`.
- Caching (safe only): in-memory, per-service, short-lived (`CacheEntry { value, fetchedAt, version }`) for categories, public config, home content, service categories. Never cache orders, payments, carts, notifications, or any admin queue beyond the current route's state.

11.2 Query/pagination contract

- `CursorPage<T> { items: T[]; nextCursor: string | null; limit: number }` built from `meta.pagination`.
- Catalog pagination mode (cursor vs page number) is a CONTRACT TO VERIFY. `PaginationAdapter` hides it: the catalog store exposes `items`, `hasMore`, `loadMore()`, and if the backend uses pages, also `page`. Use "Load more" on mobile catalog (Design System allows either; DESIGN.md says choice is implementation).
- Admin queues use cursor paging with "Load more" plus a refresh action; do not offer jump-to-page.

11.3 Frontend API services

Every service lists endpoints from the backend plan. Paths marked ◇ are not in the backend's endpoint table: CONTRACT TO VERIFY before use.

Frontend service	Responsibility	Endpoints (method path)	Request → response (domain)	Auth / ownership	Cache
AuthApi	session lifecycle	POST /auth/register , /auth/login , /auth/refresh , /auth/logout , /auth/verify-email , /auth/forgot-password , /auth/reset-password	credentials DTOs → SessionSummary ; neutral void	Guest / cookie / token	none
MeApi	profile	GET /me ; ◇ profile update	→ User { role, name, email, phone, emailVerified?, permissions? }	Authenticated	session-scoped in AuthStore
AddressesApi	saved addresses	◇ /addresses CRUD	Address (governorate, city, area, street, buildingNumber, floor, apartment, landmark, notes, recipientName/Phone, isDefault; no coordinates)	Owner	route store
CatalogApi	categories and products	GET /categories , GET /products , GET /products/:slug	ProductFilters +cursor → CursorPage<ProductCard> ; slug → ProductDetail	Public	categories: short TTL in memory
SearchApi	search	GET /search (q + documented filters)	→ CursorPage<ProductCard> + active filters	Public	none
ContentApi	home modules	GET /content/home	→ ContentModule[] (date- windowed by backend)	Public	short TTL
ConfigApi	public settings	◇ public config for contact/WhatsApp, business hours, payment methods list, serviceability copy	→ AppConfig	Public	TTL; refresh on focus
CartApi	cart	GET /cart , POST /cart/items , PATCH /cart/items/:itemId , DELETE /cart/items/:itemId	{productId, variantId?, quantity} / {quantity, expectedVersion} → Cart { items, version }	Guest session or customer	CartStore

Frontend service	Responsibility	Endpoints (method path)	Request → response (domain)	Auth / ownership	Cache
CheckoutApi	estimate	POST /checkout/shipping-estimate	fulfillment + address → ShippingEstimate { estimateMinor?, serviceable } or SHIPPING_CONFIGURATION_UNAVAILABLE	Guest/customer	none
OrdersApi	orders	POST /orders, GET /orders/:reference, PATCH /orders/:reference, POST /orders/:reference/cancel, ◇ COD confirm, ◇ list "mine"	create DTO (+idempotency) → OrderCreated { reference, status, totals, guestToken? }; detail → Order	Guest token / owner	none
PaymentsApi	payment info and proof	GET /orders/:reference/payment, POST /orders/:reference/payment-proof/upload-config, POST /orders/:reference/payment-proofs, ◇ history list	→ PaymentInfo { methodSnapshot, instructions, status, proofRequired, proofs[] }	Owner	none
PreOrdersApi	pre-orders	POST /products/:slug/pre-orders, GET /pre-orders, ◇ detail, ◇ customer pay/confirm	{variant, quantity, contact} → PreOrder	Guest/customer; owner	none
ReturnsApi	returns/refund view	POST /orders/:reference/returns, ◇ detail GET	{items[{orderId, quantity, reason}], evidenceMetadata?} → ReturnRequest + Refund?	Owner	none
ServiceCatalogApi	service config	GET /services, GET /services/:slug	→ ServiceCategory { slug, name, description, fields[], communicationChannels[] } (fields may be empty: OD-12)	Public	short TTL

Frontend service	Responsibility	Endpoints (method path)	Request → response (domain)	Auth / ownership	Cache
ServiceRequestsApi	request lifecycle	POST /services/:slug/requests , GET /service-requests/:reference , POST /service-requests/:reference/quotation/accept , .../reject , ◇ list "mine"	→ ServiceRequest , Quotation	Guest/customer submit; owner	ServiceRequestStore
NotificationsApi	persisted notifications	◇ list, ◇ mark read, ◇ unread count	→ CursorPage<Notification>	Owner	NotificationStore
AdminOrdersApi	order operations	GET /admin/orders , POST /admin/orders/:reference/accept	reject	status	shipping
AdminPaymentsApi	proof review	◇ queue, ◇ signed-URL fetch, POST /admin/payments/:paymentId/confirm	reject	request-new-proof`	{expectedVersion, reason?/note?} → Payment
AdminCatalogApi	products/categories	POST /admin/products , PATCH /admin/products/:id , POST /admin/categories , ◇ lists, ◇ category PATCH	product DTO + version	products.write , categories.write	route store
AdminInventoryApi	stock	POST /admin/inventory/:productId/adjust , ◇ list	{variant, delta, reason, expectedVersion}	inventory.write	route store
AdminServicesApi	service ops	POST /admin/service-requests/:reference/quotes , ◇ queue/detail, ◇ catalog edit, ◇ status progression	{amountMinor, currency, note} → Quotation	services.quote	route store
AdminPreOrdersApi		POST /admin/pre-orders/:reference/accept , ◇ queue, ◇ reject, ◇ mark available	{expectedVersion, availabilityDate?}	preorders.write	route store
AdminShippingApi		POST /admin/orders/:reference/shipping , ◇ queue	{provider, finalCostMinor, reason, expectedVersion}	shipping.write	route store

Frontend service	Responsibility	Endpoints (method path)	Request → response (domain)	Auth / ownership	Cache
AdminReturnsApi	returns/refunds	`POST /admin/returns/:reference/ap prove	reject , POST /admin/refunds/:refundId/co mplete` , ◇ queue/detail	`{note	reason, expectedVersion} / {amount Minor, method, timestamp, internalReference}`
AdminCustomersApi		◇ list/detail		CONTRACT TO VERIFY	route store
AdminContentApi		◇ modules CRUD, product description edit via PATCH /admin/products/:id		CONTRACT TO VERIFY	route store
AdminCouponsApi		◇ CRUD, usage	value is basis points for percentage and minor units for fixed (MongoDB plan \$5.19)	CONTRACT TO VERIFY	route store
AdminReportsApi		GET /admin/reports/:report	date/status/geography filters → report rows	reports.read	none
AdminAuditApi		GET /admin/audit-logs	entity/actor/date filters → AuditEntry[]	audit.read	none
AdminSettingsApi		PATCH /admin/settings/:key , ◇ read	typed value + version	settings.write or Owner	route store

Do not create one API service per backend file. The table above is the frontend domain split; the Admin services intentionally mirror admin modules because their permission scopes differ.

11.4 Error behavior per call (summary; full strategy in §33)

Reads: show the section's error state with `Retry`. Mutations: keep inputs, show field errors from `error.fields`, show a banner for business errors, and for `409` reload the record and show "this record changed, review and retry".

12. State Management

12.1 Decision

No `NgRx`. The sources do not justify a global event-sourced store: the domains are mostly request/response with a handful of shared pieces. Use signals-based feature stores (plain injectable classes), `RxJS` at the HTTP/stream boundary, and `computed()` for derived data. Feature stores are provided on the route (`providers: [FooStore]`) so they are destroyed when the user leaves.

12.2 Where state lives

Kind	Owner	Lifetime	Examples
Server state (read models)	Route-scoped stores, fetched via API services	Route subtree	<code>CatalogStore</code> , <code>OrderDetailStore</code> , <code>AdminOrdersStore</code>
Session state	<code>AuthStore</code> (global)	App	user, role, status, access token holder
Cart state	<code>CartStore</code> (global)	App	cart lines, version, <code>status</code> , last change set
Notification state	<code>NotificationStore</code> (global)	App (customer, and admin variant)	list, unread count, last-synced
App config	<code>AppConfigStore</code> (global)	App	contact/WhatsApp, payment-method list, feature flags
Locale/direction	<code>LocaleService</code> (global)	App, persisted	<code>ar / en</code> , <code>dir</code>
UI state	Component-local signals or tiny UI stores	Component	open drawer, selected tab, mobile filters sheet
Draft state	<code>CheckoutStore</code> (route-scoped to <code>/checkout/**</code>)	Until order success	contact, address, fulfillment, method, coupon, proof files metadata, idempotency key
URL state	Router query params	Navigation	catalog filters, <code>q</code> , <code>variant</code>

Rule: if the URL can hold it and a shared link should reproduce it, it lives in the URL (catalog filters). If the backend owns it, the store holds a cached copy with `loadedAt` and `version`, never an independent truth.

12.3 Store shape (all feature stores)

Each store exposes: `state` (a readonly signal object), `status` (`idle | loading | refreshing | success | error`), `error` (`ApiError | null`), `data` computed accessors, and action methods returning `void` or a promise/observable result. Mutations expose `pending` / `lastResult` signals so UI can show mutation pending/success/failure (§34). No store imports another feature's store; cross-feature signals use global stores.

12.4 Effects (side-effect rules)

- Auth changes → reset cart cache, clear notifications, reconnect socket (an `effect` in `core/auth`).
- Locale change → update `<html lang dir>`, refetch localized public reads (catalog list keeps filters).
- Realtime event → call the owning store's `revalidate()`; never write event payload straight into authoritative state except a minimal optimistic marker allowed by \$20.
- Effects live in services/stores, not in components.

12.5 Anti-patterns to avoid

A single mega store; storing DTOs in state; subscribing in components without

`takeUntilDestroyed` / `toSignal`; duplicating the same list in two stores; local-storage-backed server state; computing totals client-side.

13. Internationalization / RTL / LTR

Topic	Decision
Locales	<code>ar</code> (default, <code>dir="rtl"</code>), <code>en</code> (<code>dir="ltr"</code>). One build.
Library	One runtime i18n layer (§6). Dictionaries split by scope: <code>common</code> , <code>storefront</code> , <code>checkout</code> , <code>services</code> , <code>account</code> , <code>admin</code> , <code>errors</code> , <code>status</code> . Admin dictionary is lazy-loaded with the admin bundle. English admin strings may be incomplete at launch; fallback to Arabic key text, never to a raw key (§31 of the Design System: Arabic is primary).
Keys	Semantic (<code>checkout.payment.proofRequired</code>), never sentence-as-key. No literal UI string in templates or components. ESLint rule bans text nodes in templates except symbols.
Backend text	Localized content arrives as <code>{ ar, en? }</code> . A <code>localized()</code> pipe picks the active locale and falls back to <code>ar</code> . Backend error <code>message</code> is not shown directly; the UI maps <code>error.code</code> to its own translation and uses <code>message</code> only in the technical-details disclosure.
Direction	<code>DirectionService</code> sets <code>document.documentElement.lang/dir</code> on boot and on switch; the SSR-rendered HTML carries the same. All SCSS uses logical properties (<code>margin-inline</code> , <code>padding-inline</code> , <code>inset-inline</code> , <code>text-align: start</code> , <code>border-start-start-radius</code>). Stylelint forbids <code>left/right/margin-left/margin-right/padding-left/padding-right</code> outside a documented allowlist (physical-side drawers, WhatsApp float if fixed).
Icons	Direction icons (back, forward, chevrons, arrows, breadcrumb separators) flip via <code>:dir(rtl)</code> CSS transform or an <code>IconDirective</code> flag; object icons (search, cart, book, printer, upload, WhatsApp) never flip.
LTR islands	<code><bdi> / dir="ltr"</code> wrapper component <code>LtrText</code> for phone numbers, emails, ISBNs, URLs, order references, coupon codes. Phone and email inputs use <code>dir="ltr"</code> while the field stays positioned by the page direction.
Persistence	Locale stored in a first-party cookie or <code>localStorage</code> through <code>SafeStorage</code> (cookie preferred if SSR is adopted so the server can render the right direction). Default <code>ar</code> .
Route strategy	The IA defines no locale prefix. FRONTEND DECISION: no locale in the URL. Consequence: English pages are not separately indexable and no <code>hreflang</code> can be emitted. Whether English needs SEO is FB-14 (OPEN DECISION).

Topic	Decision
Numbers/currency	<code>Intl.NumberFormat(locale, { style: 'currency', currency: 'EGP' })</code> from <code>amountMinor / 100</code> . Minor-unit factor: the MongoDB plan says "if the configured EGP minor-unit convention is piastres" (100). The factor lives in one constant <code>MINOR_UNITS_PER_EGP</code> ; confirm with the backend (CONTRACT TO VERIFY). Do not mix numeral styles in one amount. Arabic-Indic vs Latin digits for prices and dates is OPEN (Q-04 in DESIGN.md): FB-13. Implement <code>numberingSystem</code> as a config value, default to the locale's <code>Intl</code> default until decided.
Dates	<code>Intl.DateTimeFormat</code> with <code>timeZone: 'Africa/Cairo'</code> ; the backend sends UTC. Relative times ("5 minutes ago") only for notifications, with absolute time in a tooltip/ <code>title</code> .
Plurals	ICU plural support in the library for Arabic (zero/one/two/few/many/other).
Validation messages	Keyed by validator code in the <code>errors</code> scope (§21).
Mixed content	Product titles may mix Arabic and Latin; never concatenate strings in code; use interpolation keys and <code>LtrText</code> for embedded technical tokens.
Accessibility	<code>lang</code> attribute on inline foreign-language fragments; screen-reader labels translated, including icon-only buttons.

14. Design System Implementation

Source: DESIGN-SYSTEM.md and DESIGN.md (values, rules), UX-UI-SPECIFICATION.md (components), `design-website/` mockups (Gap A). Values below are the Design System's proposal; its palette/fonts are marked "verify against approved logo" (Q-03/FB-12); keep them as tokens so a correction is a single edit.

14.1 Token architecture

Three tiers in `styles/tokens`, all exposed as CSS custom properties on `:root`:

1. Primitive (raw): `--color-brand-green-700: #0f4c3a`, `--space-4: 16px` ...
2. Semantic (role): `--color-primary`, `--color-surface`, `--color-text-primary`, `--color-border`, `--color-focus-ring`, `--color-success/warning/error/info` (+ `--soft` backgrounds).
3. Component (optional, only where needed): `--button-height-md: 44px`, `--input-height-md: 44px`, `--card-padding-mobile: 16px`, `--card-padding-desktop: 20px`, `--product-image-ratio: 2 / 3`, `--focus-ring-width: 2px`.

SCSS mirrors them with maps/mixins (breakpoint mixin, `type()` mixin) so templates never use raw hex or px for design values.

Group	Values to implement (from the Design System)
Color	green 900 <code>#083328</code> , 800 <code>#0B3F31</code> , 700 <code>#0F4C3A</code> (primary), 600 <code>#17624C</code> , 100 <code>#E6F0EC</code> ; gold 700 <code>#80601D</code> (text-safe), 600 <code>#A47925</code> , 500 <code>#B58B3A</code> (decorative only), 100 <code>#F5EDD9</code> ; ivory 50 <code>#FCFBF8</code> (canvas), 100 <code>#F6F4EE</code> ; white; text <code>#1F2A26</code> / <code>#53615B</code> / <code>#718078</code> (placeholder only); border <code>#D7DED9</code> / <code>#AEBBB4</code> ; success <code>#176B4D</code> / <code>#E5F3EC</code> ; warning <code>#8A5A00</code> / <code>#FFF4D6</code> ; error <code>#B42318</code> / <code>#FDE9E7</code> ; info <code>#155E75</code> / <code>#E6F4F7</code>

Group	Values to implement (from the Design System)
Typography	Arabic: IBM Plex Sans Arabic; Latin/numerals: Inter; fallbacks system-ui. Scale: display-xl 48/1.12 (m 36), display-lg 40 (m 32), h1 32/1.25 (m 28), h2 26/1.3 (m 22), h3 22/1.35 (m 20), h4 18/1.4 (m 17), body-lg 18/1.7, body 16/1.65, body-sm 14/1.55, caption 12/1.45, label 14 600, button 15 600, price-lg 24, price 18, product-title 16 600, table 14/1.5, form-error 13, helper 13. Tabular numerals for prices/ids/counts. Arabic never justified, never uppercase
Spacing	4px base: 4, 8, 12, 16, 20, 24, 32, 40, 48, 64, 80, 96, 128
Radius	sm 6 (inputs), md 10 (buttons, cards, product image container), lg 16 (panels, modals), xl 24 (hero only), pill 999 (badges/chips)
Elevation	border first; shadow-sm/md/lg/modal/floating as in the Design System (§14), rare on grid cards
Motion	fast 120ms, normal 200ms, slow 320ms; <code>cubic-bezier(.2,.8,.2,1)</code> for drawers/modals; <code>prefers-reduced-motion</code> removes transforms and shimmer
Z-index	base 0, sticky 20, dropdown 40, drawer 60, modal 80, toast 100
Breakpoints	xs 0–479, sm 480–767, md 768–1023, lg 1024–1279, xl 1280–1439, 2xl 1440–1919, 3xl 1920+ (validate at 375, 480, 768, 1024, 1280, 1440, 1920)
Containers	reading 720, content 1120, wide 1280, max 1440; grid 12/8/4 columns, gutters 24/20/16
Dark mode	Not in V1 (Design System recommendation). Keep tokens semantic; do not ship a toggle.

14.2 Global vs feature styles

- Global (`styles/`): tokens, reset, typography, focus ring, direction helpers, container/grid utilities, `.visually-hidden`, skip link, print-safe basics.
- Component styles: every `shared/*` component owns its SCSS (view encapsulation default). They consume semantic tokens only.
- Feature styles: only layout composition specific to a page (e.g. checkout two-column split). A feature must not restyle a shared component. If it needs a new look, add a variant to the shared component with a documented reason.
- No `!important`, no deep selectors (`::ng-deep`) except in one documented place for CDK overlay panes.
- Geometric Islamic motif: one optional SVG background component (`BrandMotif`), 4–8% opacity, used only on the home hero and empty states, never behind forms/tables/text (Design System §6.3).

14.3 Primitive inventory (Design System §21 → Angular)

`Button` (primary, secondary, tertiary, destructive, loading, disabled; 44px min on mobile), `IconButton` (needs accessible name + tooltip), `Link`, `Badge`, `StatusBadge`, `Alert`, `Card / Surface` (flat/card/raised), `Tabs`, `Breadcrumbs`, `Pagination` or `LoadMore`, `Skeleton`, `EmptyState`, `ErrorState`, `Spinner`, `Tooltip`, `Divider`, `Avatar`, `Input`, `Select`, `Checkbox`, `Radio`, `Switch`, `Textarea`, `QuantityStepper`, `FileDropzone`, `Dialog`, `Drawer`, `BottomSheet`, `Toast`, `Menu`, `DataTable`, `Timeline`.

Do not create a second variant of anything above without a written reason. Status color is never the only signal: `StatusBadge` always renders text + (where useful) icon + tone.

15. Shared Components

Definition of each (responsibility → inputs → notes). All are standalone, OnPush, and take semantic inputs, not style flags.

Component	Responsibility	Key inputs / outputs	Notes
<code>StatusBadge</code>	Render any status	<code>`kind: 'order'`</code>	'payment'
<code>Timeline</code>	Chronological lifecycle	<code>steps[]</code> , <code>current</code> , <code>terminal?</code>	Vertical on mobile; chronological in RTL; terminal states stay visible
<code>PriceDisplay</code>	Current price + optional original (discount)	<code>money</code> , <code>originalMoney?</code> , <code>size</code>	Tabular numerals; never computes totals
<code>AvailabilityBadge</code>	In stock / Out of stock / Pre-order available	<code>availability</code>	Text + icon + tone
<code>ProductCard</code>	Cover (2:3, <code>object-fit: contain</code>), title, optional author/grade/publisher (only when present), price, availability, action	<code>product</code> , <code>action</code> auto-derived from availability	Border, not shadow; hover = border/surface only. Missing image → calm placeholder with localized "Image coming soon"
<code>ProductGrid</code>	Responsive grid (1-2 columns mobile, 3 tablet, 4 desktop)	<code>items</code> , <code>loading</code>	Skeleton cards while loading; stable card height
<code>ProductGallery</code>	Full-cover gallery; thumbnails only if >1 image; swipe + indicator + accessible prev/next	<code>images</code>	No hover zoom that hides cover
<code>VariantSelector</code>	Pick variant attributes	<code>variants</code> , <code>selected</code> , <code>(select)</code>	Updates price/availability; required before add-to-cart when variants exist (PRODUCT RECOMMENDATION; the backend also validates)
<code>QuantityStepper</code>	Integer ≥ 1	<code>value</code> , <code>max?</code>	Pending state; no negative
<code>CartLine</code>	One cart row	<code>line</code>	Shows availability warning, remove
<code>OrderSummary</code>	Items + totals from server	<code>totals</code>	Shows estimate vs final shipping separately
<code>EstimateVsFinalCost</code>	Distinguish shipping estimate from Admin final cost	<code>estimate</code> , <code>final?</code> , <code>bindingKnown</code>	Never says the customer accepted an adjustment (OD-04)
<code>CouponField</code>	Apply/remove coupon	<code>(apply)</code> , <code>result</code>	Message from backend result mapped to a translation; no stacking/expiry copy (OD-17)
<code>PaymentProofUploader</code>	Select, validate, preview, upload, retry	see §29	Only in non-COD flows
<code>WhatsAppCta</code>	Contextual support link	<code>`context: 'global'`</code>	'product'
<code>ContactBlock</code>	Address, hours, social, phone	from <code>AppConfig</code>	Shows only confirmed data; phone only after OD-01; missing → hidden, not invented
<code>DataTable</code>	Admin tables	<code>columns</code> , <code>rows</code> , <code>sort?</code> , <code>selection?</code>	Mobile: priority-card fallback or scroll region with kept headers; actions never hover-only
<code>OperationalQueueTable</code>	Queue tabs + rows + permission-aware actions		Extends <code>DataTable</code>

Component	Responsibility	Key inputs / outputs	Notes
<code>ConfirmDialog</code>	Destructive/terminal confirm	<code>title</code> , <code>consequence</code> , <code>confirmLabel</code> , <code>tone</code>	CDK dialog, focus trap/restore
<code>Drawer</code> / <code>BottomSheet</code>	Filters/navigation		Bottom sheet on mobile for filters
<code>Toast</code>	Short, non-critical confirmations only		Never the only place for a critical result
<code>Alert</code>	Persistent contextual message	<code>tone</code> , <code>icon</code> , <code>body</code>	Used for payment, shipping, availability, quotation notices
<code>EmptyState</code> / <code>ErrorState</code> / <code>Skeleton</code>	State patterns		<code>ErrorState</code> shows retry, safe navigation, optional WhatsApp, collapsible request ID
<code>FormField</code>	Label + control + helper + error wiring		<code>aria-describedby</code> , visible label always
<code>ErrorSummary</code>	Top-of-form list for long forms		Focus on first error on submit only
<code>Breadcrumbs</code> , <code>Tabs</code> , <code>LoadMore</code> , <code>Pagination</code>	Navigation		Separators mirror in RTL
<code>LtrText</code>	Isolate LTR strings		Phone, email, ISBN, reference
<code>BrandMotif</code>	Decorative geometry		<code>aria-hidden</code>

Common page contract (applies to every page spec in §16–§19)

To keep the page tables readable, these requirements apply to every page unless the row says otherwise. A page is not done until it satisfies them (§39).

- Layout: inside its shell; `<main id="main">` landmark, one `<h1>`, skip link; focus moves to `<h1>` after route change; announce route title.
- States: initial loading (skeleton matching final structure), refreshing (keep content, small inline indicator), empty (`EmptyState` with next step), partial data (render what loaded; failed section shows its own `ErrorState` with `Retry`), API error (`ErrorState` with `retry`; mutations keep inputs), permission denied (`/403` or in-place "no permission"), not found (`/404`, no data leakage), mutation pending (action disabled + spinner, label kept), mutation success (inline result and/or toast; precise wording: "Pending Review", not "Done"), mutation failure (field errors + banner).
- Responsive: mobile-first; one primary action per screen on phones; touch targets $\geq 44\text{px}$ (customer) / $\geq 40\text{px}$ (dense admin icon actions with spacing); verify at 375/480/768/1024/1280.
- RTL/LTR: logical properties only; direction icons flip; LTR islands for technical strings.
- A11y: labels, `aria-describedby` errors, `aria-live` for async results, keyboard-complete, visible focus, reduced motion.
- SEO: public indexable pages set title/description/canonical via `SeoService`; private pages emit `noindex`.
- Analytics (only events the PRD/UX supports; §37 of UX spec): search submitted, filter applied/cleared, product viewed, add to cart, pre-order requested, checkout started, fulfillment selected, payment method selected, payment proof submitted, order submitted, order cancelled, service request submitted, quote accepted/rejected, WhatsApp clicked, return requested. No external analytics dependency; the facade defaults to a no-op until an approach is approved. No PII, tokens, or proof data in events.

- Design source: each page's row names the expected `design-website/` item; see Gap A.
-

16. Storefront Architecture

16.1 Shell

`StorefrontShell`: `StoreHeader` (desktop: logo, Shop, Services, Search, Account, Cart, WhatsApp; mobile: menu, logo, search, cart, account in menu, persistent WhatsApp that must not cover primary actions or the checkout total), `<router-outlet>`, `StoreFooter` (confirmed info only). `CartMini` reads `CartStore.count`. Header search is a labelled form that navigates to `/search?q=`.

16.2 Page specifications

Columns: Page / route | Purpose & role | Components | API calls | Local state & URL params | Page-specific states, validation, navigation | PRD refs · design source

Page / route	Purpose & role	Components	API calls	Local state & URL	Page-specific behavior	Refs · design
Home <code>/</code> (PUB-001)	Front door; Any	Hero (Shop CTA), book discovery shelf, category cards (Books, School/Study, Other), seasonal <code>ContentModule</code> , services preview, trust strip, <code>WhatsAppCta</code> , footer	<code>GET /content/home</code> ; <code>GET /categories</code>	none	Hero copy: no "best" claims; copy is TBC (Q-12 DESIGN.md). If no seasonal module is active the page still looks complete. Content error → show static structure (categories) and an inline retry for modules. Seasonal modules link to existing catalog/service routes only	CMS-001-004, CAT-004-009 · <code>design-website/home</code>
Shop landing <code>/shop</code> (PUB-002)	Permanent catalog entry; Any	category cards (Books first), featured products, <code>ProductGrid</code>	<code>GET /categories</code> , <code>GET /products</code> (featured via content or default order)	none	Empty catalog → explanation + Contact/WhatsApp	CAT-001-009 · <code>shop</code>
All products <code>/shop/all</code> (CAT-001)	Browse; Any	<code>SearchBar</code> , <code>FilterPanel</code> , <code>ActiveFilters</code> , <code>ProductGrid</code> , <code>LoadMore</code>	<code>GET /products</code>	filters in URL (§22)	no products / no results / error states; filter drawer on mobile	CAT-001-009, SEARCH-001-005 · <code>catalog</code>
Category <code>/categories/:slug</code> (CAT-002)	Stable category listing	same + category header	<code>GET /categories</code> (resolve slug), <code>GET /products?category=</code>	URL filters	unknown slug → 404; empty category → parent categories/search	same · <code>category</code>
Search <code>/search</code> (CAT-003/004)	Find by name, author, grade, stage, subject, publisher, ISBN, category	same	<code>GET /search</code>	<code>q</code> + filters	initial (no query), results, no results (repeat query+filters, clear/refine, Shop, WhatsApp), API error. Validate query length per backend (CONTRACT TO VERIFY); trim only. No autocomplete	SEARCH-001-005 · <code>search</code>

Page / route	Purpose & role	Components	API calls	Local state & URL	Page-specific behavior	Refs · design
Product detail /products/:slug (CAT-005)	Evaluate + purchase/pre-order; Any	Breadcrumbs , ProductGallery , title/metadata list (only populated fields), VariantSelector , PriceDisplay , AvailabilityBadge , QuantityStepper , primary action, shipping/pickup info (delivery estimated; pickup free; no delivery-time promise), description (or calm no-description state), WhatsAppCta(product)	GET /products/:slug ; POST /cart/items	variant query param; selected variant, quantity	In stock: require variant if variants exist, then Add to Cart (success toast + cart count; errors INSUFFICIENT_STOCK / AVAILABILITY_CHANGED → inline). Out of stock + eligible: no Add to Cart; Pre-order → /products/:slug/pre-order . Out of stock + not eligible: clear message, no CTA, WhatsApp + browse. Not found → 404. No reviews, ratings, or related-products block (Q-06 open: do not build). SEO: title/description/availability meta	PDP-001-005, VAR-001-004, PRE-001 · product
Pre-order request /products/:slug/pre-order (PRE-001)	See \$26	PreOrderForm , product summary, notice (no date promised)	GET /products/:slug , POST /products/:slug/pre-orders	variant, quantity	\$26	PRE-001-007 · product (verify)
Cart /cart (TRX-001)	Review fixed-price items	CartLine , QuantityStepper , CouponField ? , totals preview, Proceed, WhatsAppCta(cart)	GET /cart , PATCH/DELETE /cart/items/:id	none	\$23	CART-001-005 · cart
Checkout steps /checkout/*	\$23				\$23	CHK-001-012 · checkout

Page / route	Purpose & role	Components	API calls	Local state & URL	Page-specific behavior	Refs · design
Order confirmation /orders/:reference/confirmation (TRX-007)	Receipt + reference + "Pending Review" + next step + "save your order link" (guest)	OrderSummary , Timeline (first state), WhatsAppCta(order)	GET /orders/:reference	guest token handling \$10.5	Never says "confirmed"/"paid". Next-step text depends on payment method/fulfillment (from server). Refresh-safe: reload reads persisted order	ORD-001-009, CHK-012 · order-confirmation
Contact /contact (SUP-001)	Confirmed contact/location	ContactBlock , WhatsAppCta(global)	ConfigApi	none	Missing phone/WhatsApp (OD-01) → show only confirmed info; landmark wording TBC (Q-02): show whatever config contains, never hard-code	WA-001-003 · contact
404 / 403	Safe recovery	ErrorState	none	none	Links: Home, Shop, Search, Login	— · error

Header search and WhatsApp: `WhatsAppCta` URL comes from config (OD-01). If config is absent the button is hidden, not given an invented number.

17. Customer Dashboard Architecture

`AccountShell` (desktop: compact sidebar; mobile: stacked section menu; active section explicit; breadcrumbs *Account* → *section* → *detail*; current orders/services/notifications reachable in one tap). Guard `authGuard('customer')`. Only the signed-in customer's own records are ever requested.

Page / route	Purpose	Components	API	State / URL	Page-specific behavior	Refs · design
Overview <code>/account</code> (ACC-006)	Current activity first	order/service/notification summary cards, unverified-email banner (if <code>/me</code> says so; FB-05)	◇ my orders (recent), ◇ my services (recent), notifications unread	<code>AccountOverviewStore</code>	Partial data allowed: each card loads and fails independently. Empty cards link to Shop / Services	AUTH-001-012 · <code>account</code>
Profile <code>/account/profile</code> (ACC-007)	View/edit profile	<code>ProfileForm</code>	<code>GET /me</code> , ◇ update	form	Validation as register (name required; phone canonical; email unique → server <code>409</code>). Changing email → follows backend (verification re-required? CONTRACT TO VERIFY). Never shows password hash/tokens	AUTH · <code>profile</code>
Orders <code>/account/orders</code> (ACC-008)	History + current state	order list rows (<code>StatusBadge</code> , reference, date, total, next action)	◇ my orders (cursor)	<code>status</code> filter optional (only if backend supports)	Empty → Shop. Row → <code>/orders/:reference</code>	ORD · <code>orders</code>
Addresses <code>/account/addresses</code> (ACC-009)	Manage addresses	list, <code>AddressForm</code> , <code>ConfirmDialog</code> (delete)	◇ <code>/addresses</code> CRUD	edit/delete dialog state	Fields: recipient name & phone, governorate, city, area, street, building, floor, apartment, landmark, notes, default. No map pin. Governorate = select (list source CONTRACT TO VERIFY: static in app vs API; if static, it must be one list shared with checkout)	AUTH-011, CHK-002-006 · <code>addresses</code>

Page / route	Purpose	Components	API	State / URL	Page-specific behavior	Refs · design
Payment history /account/payments (ACC-010)	Payments linked to orders	table/cards: order ref, method, amount, payment status, proof status, refund link	◇ payments list		Distinguish original payment, proof, and refund record (labelled separately). Never show proof images in the customer list unless the backend explicitly exposes customer proof viewing (not confirmed; the DB plan says customer visibility is "where approved")	PAY, REF · payments
Service requests /account/services (ACC-011)	List requests with quote/payment state	rows with StatusBadge(service)	◇ my service requests		Empty → Service Catalog with the request→review→quotation explanation	SRV · account-services
Service request detail /account/services/:reference (SRV-005)	Track	Timeline(service), request fields, communication context, WhatsAppCta(service), quotation card when sent	GET /service-requests/:reference		\$27	SRV-001-008, QUOTE · service-request
Quotation /account/services/:reference/quotation (SRV-006)	Decide	QuotationCard	same + accept/reject		\$27	QUOTE-001-005 · quotation
Pre-orders /account/pre-orders (ACC-012) + detail (PRE-002)	Progress	list + detail with Timeline(preorder)	GET /pre-orders, ◇ detail		\$26	PRE · preorders
Notifications /account/notifications (ACC-013)	Persisted list	NotificationItem, mark-read	◇ list/mark-read	cursor	Title, status, reference, direct action, timestamp. Empty → "no current notifications"	NOT-001-004 · notifications
Return detail /account/returns/:reference (RET-002)	Return and refund record	Timeline(return), items, reason, refund record	◇ return detail		\$28	RET, REF · return

Page / route	Purpose	Components	API	State / URL	Page-specific behavior	Refs · design
Order detail / tracking / return request (shared with guests)					\$24, \$28	

18. Student Services Architecture

18.1 Separation rules (CONFIRMED; enforce in code review)

1. `features/services` and `domain/services` do not import from `checkout`, `CartStore`, `OrdersApi`, or `ProductCard`.
2. No route, component, or model named like "service product", "service cart", "service checkout".
3. No file input in any service form. A lint/unit test asserts the service request form template contains no `input[type=file]` and no `FileDropzone`.
4. No price is shown before an Admin quotation exists. Meta tags, cards, and detail pages never show a service price.
5. Service statuses use their own labels (`submitted`, `admin_review`, `quotation_sent`, `awaiting_payment`, `payment_verification`, `payment_confirmed`, `processing`, `completed`, `closed_not_proceeding`, `closed_declined`), not order wording.

18.2 Page specifications

Page / route	Purpose	Components	API	State / URL	Page-specific behavior	Refs · design
Services landing <code>/services</code> (SRV-001)	Explain why services differ from shopping	explainer (2–3 plain sentences), confirmed categories (printing, photocopying, binding, applications & transfers, research/formatting help), file-exchange explanation, <code>WhatsAppCta(service)</code>	<code>GET /services</code>	none	The lifecycle strip <i>Request</i> → <i>Details</i> → <i>Library Review</i> → <i>Price/Confirmation</i> → <i>Fulfillment</i> . Empty/error states. Services that exist only in the backend appear; the frontend does not hard-code categories	SRV-001-008 · <code>services</code>
Service catalog <code>/services/catalog</code> (SRV-002)	Browse	<code>ServiceCard</code>	<code>GET /services</code>	none	Same data, list layout	same · <code>services-catalog</code>
Service detail <code>/services/:slug</code> (SRV-003)	Describe one service	description, how it works, Request CTA, <code>WhatsAppCta</code> /Telegram note	<code>GET /services/:slug</code>		"Final price follows library review and quotation". No turnaround time unless backend returns a value (OD-13). For <i>applications & transfers</i> : copy states the library performs it on the customer's behalf	SRV · <code>service-detail</code>

Page / route	Purpose	Components	API	State / URL	Page-specific behavior	Refs · design
Service request /services/:slug/request (SRV-004)	Submit	DynamicServiceForm , WhatsAppCta , notice about file exchange via WhatsApp/Telegram	GET /services/:slug , POST /services/:slug/request	form	Fields rendered only from fields[] returned by the backend. Types supported by the renderer: whatever the backend field type enum defines (CONTRACT TO VERIFY); unknown type → field skipped and logged, never rendered as a guess. If fields is empty (OD-12 unresolved): show the description textarea plus contact info (as the DB plan allows description) and no invented fields. A design-review-only "fields pending business confirmation" note may appear in non- production builds. Contact for guests: name + phone (+ email) per backend contract (CONTRACT TO VERIFY). Success → shows reference, status Submitted/Admin Review , next step, and sign-in path (FB-09)	SRV-001-008 · service- request
Customer pages	see §17					

18.3 Dynamic form renderer

`DynamicServiceForm` builds a typed `FormGroup` from `ServiceFormField[]` (`key`, localized `label`, `type`, `required`, `options[]`, `active`). Only `active` fields render. `formVersion` is sent back if the backend requires it (CONTRACT TO VERIFY). Server `fields[].path` errors map to controls by `key`.

19. Admin Dashboard Architecture

19.1 Shell

`AdminShell`: persistent sidebar on desktop; compact top bar + drawer on mobile. Sidebar groups in this order (IA §12): Operations (Dashboard, Orders, Payments) · Catalog (Products, Categories, Inventory) · Services · Pre-orders · Shipping · Returns & Refunds · Customers · Content · Coupons · Reports · Settings (Settings/RBAC, Audit). Top bar: role badge, notification bell (live count), contextual WhatsApp actions.

The sidebar shows items according to `permissions` when known, otherwise by role. A hidden item is hidden; a visible but unavailable action shows disabled-with-reason; a `403` shows the forbidden state with safe navigation (Design System §26.5).

19.2 Admin conventions (apply to every Admin page)

- Queues: tabs by state, search/filter where the backend supports it (scope visible), cursor "Load more", a manual Refresh, timestamp and next action per row, empty/loading/error/forbidden variants. No invented KPI thresholds or targets.
- Detail pages: header with reference + `StatusBadge` + version; sections for items, customer, payment, fulfillment, history/audit-safe summary; action bar with only the state-appropriate actions.
- Mutations: every call sends `expectedVersion`. On success replace the record from the response (not by local patching). On `409` reload and show "this record changed; review again". On `403` show no-permission. On `422` show the business message from the code map.
- Confirmations: terminal, destructive, or financial actions only (reject order, reject/refund, cancel, delete, confirm payment, stock adjustment). Not every action.
- Reasons: reject/request-new-proof/status changes accept a reason field where the backend contract takes one (the PRD says "reason where captured"); required-ness comes from the backend, not the UI.
- Dirty-form protection on editors (`unsavedChangesGuard`).
- Real-time: queue pages subscribe to `admin.queue_changed`; the response is a *revalidate* call. Never insert socket payloads into rows directly.
- Audit display: actor, action, timestamp, resulting state, only for authorized roles; never expose raw proof URLs or PII beyond the backend's response.

19.3 Page specifications

Page / route	Purpose	Components	API	State / URL	Specifics	Refs · design
Dashboard <code>/admin</code> (ADM-001)	Operational overview and priority queues	<code>DashboardCard</code> (count + link), recent activity	counts: CONTRACT TO VERIFY (derive from queue endpoints or reports; do not invent a dashboard endpoint)		Cards show current counts only; no targets. Each card is a link to the relevant queue	ADM-001 · <code>admin-dashboard</code>
Orders queue <code>/admin/orders</code> (ADM-002)	Pending Review, payment, fulfillment queues	queue tabs, <code>OperationalQueueTable</code>	<code>GET /admin/orders?status&search&cursor</code>	<code>status</code> , <code>q</code>	Row: reference, customer snapshot, total, fulfillment, payment state, status, age	ORD, ADM · <code>admin-orders</code>

Page / route	Purpose	Components	API	State / URL	Specifics	Refs · design
Order detail /admin/orders/:reference (ADM-003)	Review and act	items + stock context, customer, payment (via \$25 review), fulfillment panel (EstimateVsFinalCost , provider select), Timeline , action bar, WhatsAppCta(admin-customer)	◇ detail; accept/reject/status/shipping/cod-reminder		Actions by status (IA \$6): Pending Review → Accept / Reject; Accepted → (digital) await payment / (COD) await customer confirmation + send reminder; Payment Confirmed or Confirmed → Preparing; Preparing → Ready for Pickup (pickup) or Shipped (delivery); Ready for Pickup → Picked Up; Shipped → Out for Delivery → Delivered; Picked Up/Delivered → Completed. INSUFFICIENT_STOCK on accept: show the message and keep Pending Review; do not offer partial acceptance (OD-21). Shipping final cost form records provider + final cost + reason; whether the customer must reconfirm is OD-04, so the UI shows the final cost as "recorded by the library" and does not claim customer acceptance. Customer cancel window is not Admin-controllable	ORD-001-009, INV, SHIP · admin-order-detail
Products /admin/products (ADM-004)	List/search/publish	table, publish toggle	◇ list; PATCH /admin/products/:id			CAT · admin-products

Page / route	Purpose	Components	API	State / URL	Specifics	Refs · design
Product editor /admin/products/new , /:id/edit (ADM-005)	Create/edit	ProductForm , VariantEditor , ImageManager	POST /admin/products , PATCH /admin/products/:id	form	Required at minimum: name, category, price (or usable variant price), availability, image(s) per backend plan \$21.1; description optional and never blocks publication. Variants: independent price/availability/stock, attribute set per product. Price/availability changes are audited server-side: show a confirm for price changes. Images: product media upload flow is a CONTRACT TO VERIFY (backend validates image metadata; whether the client uploads direct-to- Cloudinary like proofs is not specified). No review/rating fields	CAT-001-009, VAR · admin-product-editor
Categories /admin/categories (ADM-006)	Stable taxonomy	list, form	POST /admin/categories , ◇ PATCH		Books-first order; future categories (Tajweed, Qira'at, etc.) appear only if the backend returns them inactive, and are not purchasable in MVP. No taxonomy mutation from content screens	CAT-004-009 · admin- categories
Inventory /admin/inventory (ADM-012)	View stock; adjust	table total/reserved/available, StockAdjustDialog	◇ list; POST /admin/inventory/:prod uctId/adjust		Adjust requires variant, delta, reason, expectedVersion. Never shows low-stock alerts (OD-09)	INV-001-008 · admin- inventory
Customers /admin/customers (ADM-007)	Operational customer record	list, detail with linked records, WhatsAppCta	◇		Privacy: only fields the backend returns	ADM · admin-customers

Page / route	Purpose	Components	API	State / URL	Specifics	Refs · design
Payments queue /admin/payments (ADM-008)	Verify proofs	queue (under_review, new_proof_requested)	◇ queue; confirm/reject/request-new-proof		§25	PAY-001-008 · admin-payments
Services /admin/services (ADM-009)	Service catalog + request queue	tabs: Requests / Catalog	◇		Catalog editing of fields[] is a configuration UI whose field definitions are owner-supplied (OD-12): build the editor generically; do not preload fields	SRV · admin-services
Service request detail /admin/services/requests/:reference (ADM-010)	Review, clarify, quote, progress	request fields, WhatsAppCta, QuotationForm, status progression	POST /admin/service-requests/:reference/quotes, ◇ detail/status		§27	SRV, QUOTE · admin-service-request
Pre-orders /admin/pre-orders (ADM-011)	Review	queue + detail	POST ../accept, ◇ reject, ◇ mark available		§26	PRE · admin-preorders
Shipping /admin/shipping (ADM-013)	Provider, estimate, status	queue of orders needing shipping action	◇ queue; POST /admin/orders/:reference/shipping		Rate source/logic is OD-03: no rate table in the UI	SHIP-001-009, PICK · admin-shipping
Returns /admin/returns (ADM-014)	Review returns	queue, detail	approve/reject		§28	RET · admin-returns
Refund record /admin/refunds/:refundId (ADM-015)	Record manual refund	RefundForm	POST /admin/refunds/:refundId/complete		§28	REF · admin-refund
Notifications /admin/notifications (ADM-016)	Operational events	list	◇		Live updates through §20	NOT · admin-notifications
Content /admin/content (ADM-017)	Descriptions + seasonal/home modules	module editor (title, body, product/category picks, start/end, order, active)	◇		Date windows; cannot change categories	CMS-001-004 · admin-content

Page / route	Purpose	Components	API	State / URL	Specifics	Refs · design
Coupons <code>/admin/coupons</code> (ADM-018)	Manage discounts	list, form	◇		Fields: type (percentage / fixed), value (percentage entered as % and sent as basis points; fixed as EGP converted to minor units, per DB plan \$5.19 — CONTRACT TO VERIFY), scope (order/product/category), active, start/end, usage limit. Stacking, limits, expiry defaults, minimums, customer restrictions are OD-17: render those inputs only if the backend exposes them, otherwise omit	COUP-001-005 · <code>admin-coupons</code>
Reports <code>/admin/reports</code> (ADM-019)	Approved dimensions	filters, tables/charts	GET <code>/admin/reports/:report</code>	date/status/geography	Report names from backend. Dimensions: orders, revenue, outside-Qena geography, payment methods, service conversion, product/category demand, pre-order demand, coupon usage. No targets/gauges (OD-18). Charts: a minimal SVG/CSS approach; adding a chart library is a decision to justify later	REP-001-002 · <code>admin-reports</code>
Audit <code>/admin/audit</code>	Audit log view	table with entity/actor/date filters	GET <code>/admin/audit-logs</code>	filters	Read-only; no edit/delete controls	PAY-007, RBAC · (no mockup expected)

Page / route	Purpose	Components	API	State / URL	Specifics	Refs · design
Settings /admin/settings (ADM-020)	Operational settings	key-based forms	PATCH /admin/settings/:key , ◇ read		Keys from the backend plan: payment.methods , payment.details , contact.whatsapp , contact.phone , shipping.* , return.policy , coupon.policy , service.policy , auth.verificationPolicy . Each key has a typed editor driven by the backend schema (CONTRACT TO VERIFY). Do not build an Owner-only screen; show what the backend permits	RBAC, OD-01-20 · admin-settings

20. Notifications & Real-Time Architecture

20.1 Principles

Persisted notifications are truth (CONFIRMED). Socket.IO accelerates delivery. Every screen must function identically without a live socket.

20.2 RealtimeService

Aspect	Rule
Creation	Browser only, lazily, after <code>AuthStore</code> becomes <code>authenticated</code> . Guests have no socket (guest order status updates via normal refresh/polling on the tracking page; polling interval is a config value, off when the tab is hidden).
Auth	Token passed in the handshake <code>auth</code> payload (backend §35.1). On token refresh, the service reads the latest token on each reconnect attempt through a callback, so reconnects never reuse an expired token. If the server rejects the handshake, treat as session issue: trigger one <code>AuthService.refresh()</code> , then reconnect once; otherwise stay offline silently.
Rooms	Whether the client joins rooms or the server auto-joins on handshake is a CONTRACT TO VERIFY. The expected rooms are <code>user:{id}</code> , <code>order:{id}</code> , <code>service:{id}</code> , and a permission-gated admin room. The client must never assume a room it is not authorized for; join failures return a generic forbidden and are ignored.
Events	<code>notification.created</code> , <code>order.status_changed</code> , <code>payment.proof_submitted</code> , <code>service.request_created</code> , <code>service.quotation_sent</code> , <code>admin.queue_changed</code> . Payloads carry a safe entity reference, <code>eventId</code> , <code>version</code> , and minimal display data.
Routing	<code>RealtimeEventRouter</code> maps event → handler: <code>notification.created</code> → <code>NotificationStore.revalidate()</code> (and a live-region announcement); <code>order.status_changed</code> → refresh the open <code>OrderDetailStore</code> /tracking if the reference matches; <code>payment.proof_submitted</code> / <code>admin.queue_changed</code> → refresh the current admin queue if relevant; <code>service.*</code> → refresh the open request/queue.
Dedupe	<code>RecentEventCache</code> of <code>eventId</code> 's (bounded, e.g. last few hundred); ignore repeats. Also ignore events whose <code>version</code> ≤ the store's version.
Reconnect	Built-in backoff. On <code>connect</code> after a drop: <code>revalidateAll()</code> for global stores (notifications, open order/service) because events may have been missed. Show a small non-blocking "live updates unavailable" hint only if disconnected beyond a threshold; no scary banners.
Disconnect/cleanup	On logout, route destroy of admin shell, and <code>DestroyRef</code> : remove all listeners, <code>disconnect()</code> , clear caches. Prevent duplicate listeners with a single registration table.
Fallback	On socket failure the app polls: <code>NotificationStore</code> refreshes on window focus and on a modest interval while the notification bell is visible; the open order/tracking page refreshes on focus. All are config-tunable. Reloading any page shows the persisted truth.
SSR	Never instantiated on the server.

Aspect	Rule
Failure of delivery	Never rolls back or implies a state change; the UI only changes after an authoritative refresh.

20.3 Notification UI

- `NotificationBell` (customer header and admin top bar): unread count from the store; opens a menu with the latest items and a link to the full list.
- `NotificationItem`: short localized title (from `type` + keys, falling back to the backend's localized title/body), current status/event, reference, direct action (`actionUrl` — validated as an internal route before navigation, §35), timestamp.
- Marking read: on open of the related record or an explicit action; optimistic UI allowed because it is non-financial and reversible, with rollback on failure.
- Toasts: only for low-importance confirmations (cart item added, copied reference, saved profile/address). Payment rejected, order rejected, shipping cost changed, quotation sent, network failure during submit use persistent alerts, page states, or notifications, never toasts alone (UX spec §29.3).
- A11y: new-notification announcement through a polite live region, rate-limited to avoid chatter.

20.4 Flow 10 (Backend event → UI)

1. Backend commits state; persists notification + outbox.
2. Worker emits `notification.created` / `order.status_changed` (if the user is online).
3. `RealtimeService` receives it, dedupes, routes.
4. Owning store revalidates via HTTP (authoritative).
5. UI updates (bell count, timeline).
6. If step 2–3 never happen, step 4 occurs on focus/poll/next navigation, and the persisted state still shows.

21. Forms & Validation

21.1 Strategy

- Typed reactive forms (`FormGroup<...>` , `NonNullableFormBuilder`). Form models are in `features/<f>/forms` , separate from DTOs. A `toDto()` mapper converts form value → request.
- Reusable controls in `shared/forms` implement `ControlValueAccessor` and render label, helper, error (`FormField`).
- Validation timing (Design System §22.2): validate format on blur or submit, not per keystroke; required on submit; preserve user input on any failure.
- Display: error under the field with icon + text, linked via `aria-describedby` ; long forms add `ErrorSummary` on submit and focus the first invalid control on submit only (not on every blur).
- Server validation mapping: `ApiError.fields[{ path, code }]` → `FormErrorMessage` sets errors on matching controls by dotted path; unknown paths go to the form-level banner. The code→message map lives in the `errors` dictionary. Do not display raw backend messages.
- Async validation: none by default. Email/phone uniqueness is enforced by the backend on submit (`409`), so no pre-check endpoint is assumed (avoids account-existence probing and an unspecified endpoint).
- Dirty/touched: admin editors warn on unsaved changes; customer forms do not.

- Double submit: buttons disable on pending; checkout and proof submission also use idempotency keys.
- Input modes: `type=email` + `inputmode=email`; `inputmode=tel`; ISBN/coupon/reference `inputmode=text` with `dir=ltr`; quantity `inputmode=numeric`.

21.2 Form inventory

Form	Fields (from sources)	Validation (frontend)	Notes
Login	identifier (email or phone), password	required	Generic failure message
Register	name, phone, email, password	required; email format; phone format (Egyptian/international; canonical rule from backend: CONTRACT TO VERIFY); password strength rules from backend (not specified: CONTRACT TO VERIFY; show the backend's field errors; do not invent a policy)	No OTP
Forgot / reset	email / new password (+ token from URL)	required; confirm field is a UI convenience only	Neutral success
Profile	name, phone, email	as register	
Address	recipientName, recipientPhone, governorate, city, area, street, buildingNumber, floor, apartment, landmark, notes, isDefault	required set per backend; landmark/notes optional	No map
Checkout customer	recipient name, phone, email (optional for guests)	name+phone required; email optional	
Checkout fulfillment	method (delivery/pickup), address (if delivery)	address required for delivery	Serviceability from server
Checkout payment	method	required	Methods from config
Payment proof	1..n image files, optional note	type/size/count (§29)	No transaction ID field
Service request	dynamic from <code>fields[]</code> + description + contact	required flags from config	No files
Quotation decision	accept; reject with optional note		Confirm dialog
Pre-order	variant (if any), quantity, contact	quantity integer ≥ 1	
Return request	item(s) with quantity, reason per item, evidence metadata (if backend accepts)	reasons list from backend config (CONTRACT TO VERIFY; confirmed policy case is wrong/damaged item)	Hidden/ineligible items per Admin flag
Admin product/variant	see §19.3	numeric ≥ 0 for price/stock; money as EGP decimal in the input, converted to minor units by <code>MoneyService</code>	
Admin category / inventory / coupon / content / settings / shipping / refund / quotation	per §19.3	numeric ≥ 0 , required reasons per backend	

22. Search / Filtering / Pagination

Source of truth is the URL. `CatalogStore` reads `ActivatedRoute.queryParamMap`, writes via `Router.navigate({ queryParams, queryParamsHandling: 'merge' })`, and fetches whenever the parsed `CatalogQuery` changes.

Aspect	Rule
Parameters	<code>q</code> , <code>category</code> (only on <code>/shop/all</code> and <code>/search</code>), <code>educationType</code> , <code>stage</code> , <code>grade</code> , <code>subject</code> , <code>publisher</code> , <code>priceMin</code> , <code>priceMax</code> , <code>availability</code> (<code>in_stock</code>)
Serialization	Multi-valued filters as repeated or comma form per backend; prices as numbers in EGP in the URL, converted to minor units for the API. Omit defaults and empties. Reset <code>cursor</code> / <code>page</code> whenever any filter or <code>q</code> changes.
Debounce	Typing in the in-page filter text inputs (publisher, price) debounces about 300ms (config); the search box submits on Enter/button (no live-as-you-type), which matches "explicit submit/search action" and "no autocomplete".
Sort	Only the sort options the backend accepts (default: books-first display order, set by the backend). Which options exist is Q-08/OPEN: render the sort control only if the backend exposes a list; otherwise omit. Do not invent "best selling", "newest" etc.
Active filters	<code>ActiveFilters</code> chips: each removable, plus Clear all; visible on mobile above results.
Filter UI	Desktop: sidebar or inline panel. Tablet: drawer or two-column panel. Mobile: bottom sheet with Apply and Close; result context (count/query) stays visible above.
Filter option values	Education type, stage, grade, subject, publisher options: source is a CONTRACT TO VERIFY (facet list in the response vs. static lists). Do not hard-code Azhar stages/grades.
Loading	First load = skeleton grid; subsequent filter change keeps the old grid with a dimmed/refreshing state to avoid layout jump; "Load more" appends. Cancel in-flight requests with <code>switchMap</code> .
Empty	No results: show the query and active filters, Clear/Refine, link to Shop, WhatsApp.
Products with missing metadata	Remain discoverable by populated fields; the card simply omits absent fields.
Result count	Show only if the backend returns it (the plan says "when efficient").
Accessibility	Result changes announced via live region ("N results" or "no results"); filter controls keyboard operable; sheet traps focus.

23. Cart & Checkout

23.1 Cart state (`CartStore`, global)

Signals: `cart` (`lines[]`, `version`, `currency`), `status`, `pendingItemIds`, `lastChangeSet`, `count` (computed), `conflicts` (merge conflicts). Lines hold display snapshots from the server (`unitPrice`, `lineTotal`, name, image, selected attributes, availability). Totals shown in the cart are the server's; the client may sum line totals only for display of an obvious preview and labels it as "estimated" until the server's checkout revalidation (FRONTEND DECISION; safer alternative: show server totals only. Prefer showing the server-provided subtotal when `GET /cart` returns one: CONTRACT TO VERIFY).

Operation	Behavior
Hydrate	On app start (guest: guest cart session; customer: user cart). A guest cart is identified by a secure session identifier whose transport is a CONTRACT TO VERIFY. SSR does not hydrate the cart.
Add	<code>POST /cart/items { productId, variantId?, quantity }</code> . If variants exist, variant required (UI blocks; backend enforces). Success → update from response, toast, announce. Failure → inline message by code. Services are rejected server-side; the UI never offers an add action on service pages.
Update qty	<code>PATCH /cart/items/:itemId { quantity, expectedVersion }</code> ; stepper debounced (about 300ms) and coalesced; 409 → refetch cart, show "cart changed".
Remove	<code>DELETE ...</code> with <code>expectedVersion</code> ; idempotent; undo is not offered.
Refresh	On route enter <code>/cart</code> , on window focus (throttled), after login, and before checkout review.
Stock/price changes	A <code>CartChangeSet</code> (changed price, availability, removed) from revalidation is rendered as an <code>Alert</code> list before the customer can proceed; the user must acknowledge/return to cart. Revalidation at checkout start is a PRODUCT RECOMMENDATION (CART-005) but the backend plan implements it at <code>POST /orders</code> and returns <code>PRICE_CHANGED</code> / <code>AVAILABILITY_CHANGED</code> : the UI handles both.
Unavailable lines	Cannot be silently purchasable: line shows "unavailable" and Proceed is disabled until removed or the backend says otherwise.
Guest → customer merge	Backend merges by <code>productId+variantId</code> with deterministic conflict handling and returns a conflict list (never silently drops). Trigger (implicit on login vs explicit call) and the conflict payload shape are CONTRACT TO VERIFY. UI: after login, if conflicts exist, show a dialog listing them for review; do not invent merge semantics (e.g. quantity caps). Never block login on cart merge failure.
Empty cart	<code>EmptyState</code> → Shop/Categories, keep WhatsApp
Minimum order	None (CONFIRMED). Do not show one.

23.2 Checkout flow (`CheckoutStore`, route-scoped to `/checkout/**`)

Cart → Customer → Fulfillment → Payment → [Payment proof] → Review → Submit → Confirmation

Draft. Holds contact, address, fulfillment, payment method, coupon code, proof metadata, idempotency key. Stored in memory and mirrored to `sessionStorage` (non-sensitive fields only; no proof URLs, no tokens). Cleared on success.

Step	Route	Content	API	Validation / rules
1 Customer	<code>/checkout/customer</code>	Recipient name + phone (required), email (optional for guests; enables confirmation email). Registered users: prefilled from <code>/me</code> and can pick a saved address later. A "Sign in" link is offered but guest checkout is never gated	none	Name+phone required
2 Fulfillment	<code>/checkout/fulfillment</code>	Delivery or Pickup (radio, equal weight). Delivery: address form (governorate, city, area, street, building number, floor, apartment, landmark, notes). Pickup: "Pickup — Free" (استلام من المكتبة — مجاناً) plus confirmed library location from config	<code>POST /checkout/shipping-estimate</code> on address change (debounced; only after required address fields valid)	Estimate labelled "estimate"; <code>SHIPPING_CONFIGURATION_UNAVAILABLE</code> or non-serviceable → neutral explanation + switch to pickup + WhatsApp (excluded-area list is OD-06, never hard-coded). No delivery-time promise (OD-05). No map pin.
3 Payment	<code>/checkout/payment</code>	<code>RadioGroup</code> of methods returned by config (OD-20). Each shows label, a plain-language description, and whether proof is needed. Payment details (account numbers etc.) shown only for methods that need transfer, from config, never hard-coded	<code>ConfigApi</code> payment methods ◇	If the list is empty or unavailable: a blocking <code>ErrorState</code> with WhatsApp, because submitting is impossible (<code>PAYMENT_METHOD_UNAVAILABLE</code>). Cash options (COD; cash at library only if the backend lists it) get the same visual weight as digital ones
4 Proof (conditional)	<code>/checkout/payment-proof</code>	See §25. Shown only when the selected method requires proof	upload-config + Cloudinary	Missing proof prevents submit (<code>PAYMENT_PROOF_REQUIRED</code>). COD shows no proof step

Step	Route	Content	API	Validation / rules
5 Review	<code>/checkout/review</code>	Items, quantities, product total, shipping (estimate), discount, final total, payment method, fulfillment, coupon field, change set alerts	revalidation: items as per the backend (the order create call); optional coupon check endpoint ◇	Cost breakdown shown before submit (PRODUCT RECOMMENDATION, treated as the safe pattern). Totals displayed are from the server response; if the review needs totals before the order exists, the source is a CONTRACT TO VERIFY (a preview/quote endpoint is not in the backend plan; do not invent one: until clarified, review shows the cart's server subtotal + the shipping estimate + any coupon result returned, labelled as "to be confirmed by the library" for anything not server-provided)
6 Submit	(button)	<code>POST /orders</code> with idempotency key; button disabled while pending	<code>POST /orders</code>	Success: clear cart state, store guest token (if any), navigate to <code>/orders/:reference/confirmation</code> . Failure: <code>PRICE_CHANGED</code> / <code>AVAILABILITY_CHANGED</code> → return to Review with the change list and require re-confirmation; <code>INSUFFICIENT_STOCK</code> → same; <code>PAYMENT_PROOF_REQUIRED</code> → Payment step; <code>VALIDATION_ERROR</code> → map to the right step; <code>RATE_LIMITED</code> → retry later; network failure → do not assume failure: re-submit with the <i>same</i> idempotency key (replay returns the original order), or check the draft's last known reference

Mobile: single column; sticky bottom summary with current total + next action; WhatsApp must not overlap it. Shipping/discount changes are never hidden inside a collapsed summary after they change (auto-expand + highlight). Desktop: about two-thirds form / one-third sticky summary.

23.3 Flows

FLOW 1 — Guest: browsing → product → cart → checkout → order. Guest cart (session) → steps 1–6 without login → `POST /orders` returns reference + status `pending_review` + guest token → `GuestOrderTokenStore` saves it → confirmation page → tracking via reference + token. Reload keeps working because the order is persisted and the token is in `sessionStorage`.

FLOW 2 — Registered: product → cart → checkout → order → history. Same, but the order is linked to the user; the confirmation is reachable via session; the order appears in `/account/orders` after the list refresh.

FLOW 4 — COD: Checkout without proof → order `pending_review` → Admin accepts → status `customer_confirmation_required` → customer sees a prominent "Confirm your order" action on order detail/tracking and a notification → customer confirms (endpoint `CONTRACT TO VERIFY`) → `confirmed` → Admin prepares. No timeout is displayed or assumed (OD-07): the state simply stays until confirmed; Admin can send a reminder (`cod-confirmation-reminder`).

24. Orders & Fulfillment

24.1 Status typing

`OrderStatus` union (backend/DB plan): `pending_review`, `accepted`, `awaiting_payment`, `payment_verification`, `awaiting_new_proof`, `payment_confirmed`, `customer_confirmation_required`, `confirmed`, `preparing`, `ready_for_pickup`, `picked_up`, `shipped`, `out_for_delivery`, `delivered`, `completed`, `rejected`, `cancelled`, `returned`. Payment status (separate): `not_submitted`, `proof_uploaded`, `under_review`, `confirmed`, `rejected`, `new_proof_requested`. Shipping status is a third field on the fulfillment snapshot. Never merge the three into one.

`StatusPresentation` (one shared map, `domain/status`) holds `labelKey`, `tone` (neutral/info/success/warning/error), `icon`, plus customer-simple stage. Tones follow the Design System §27 (e.g. Pending Review warning, Accepted info, Payment Confirmed success, Rejected error, Cancelled neutral). Simple customer stage (DESIGN.md §21) is displayed as a headline with the precise state as the detail line; the *wording* of the simple stages is Q-11/FB-13 (owner to confirm), so the stage→label keys are translation entries that can change without code. A `returned` order and any unrecognized status fall back to the neutral presentation.

24.2 Order detail page (`/orders/:reference`) — ORD-001

- Components: header (reference as `LtrText`, `StatusBadge`), `Timeline`, items, `OrderSummary` (product total, shipping with `EstimateVsFinalCost`, discount, final total, currency), payment panel (status + next step), fulfillment panel (pickup free / delivery, provider if library-selected, customer never selects provider), notifications strip, actions, `WhatsAppCta(order)`.
- Customer actions by state (CONFIRMED in IA §6 table):

State	Customer sees	Action(s)
Pending Review	submitted items, costs, review status	Cancel (<code>POST /orders/:reference/cancel</code> , with <code>expectedVersion</code>); Edit (<code>PATCH</code>) <i>only while pending review</i>

State	Customer sees	Action(s)
Accepted	accepted + next payment or COD confirmation step	none that alter financial total; no cancel
Awaiting Payment	instructions + proof status	Upload proof (\$25)
Payment Verification	"proof under review"	none (wait; support); no duplicate proof
Awaiting New Proof	rejection/request (reason where captured)	Upload new screenshot(s)
Payment Confirmed	payment accepted; preparation next	none
Customer Confirmation Required	COD prompt	Confirm
Preparing	preparing	none
Ready for Pickup / Picked Up	pickup is ready (free) / collected	none
Shipped / Out for Delivery / Delivered	delivery progress, no time promise	none
Completed	closed; return access if eligible	Request return (\$28)
Rejected / Cancelled	terminal + reason where captured	Shop / support

- Edit (pending review only): a controlled edit of editable items/details with recalculated totals and `expectedVersion` (backend \$14). The exact editable fields and the post-acceptance non-financial edits are FB-11 (OPEN: ORD-009 / backend OD-24). Safe behavior: offer Cancel only; implement Edit only for fields the backend contract lists as editable, and show nothing after acceptance.
- Cancel uses a `ConfirmDialog`; success → state `cancelled`, notification, toast + persistent state; `409 ORDER_STATE_CONFLICT` → reload and explain (the Admin may have just accepted).
- Reload/refresh always reads the persisted order.

24.3 Tracking (`/orders/:reference/tracking`) — ORD-002

Vertical `Timeline` (chronological in RTL), current state emphasized, terminal states visible, "waiting for the library"/"waiting for you" owner label per state, last updated time, WhatsApp. Live refresh via \$20; fallback refresh on focus. No delivery time estimates (OD-05).

24.4 Shipping adjustments (OD-03/04)

Display the estimate from checkout; when Admin records a final cost, show both clearly. Whether the final cost binds the customer or requires reconfirmation is open (OD-04): the UI shows "final cost recorded by the library" and links to WhatsApp; it does not add an accept/reject action for the new cost and does not state that the customer agreed. A banner (`Alert`, persistent) appears when the final differs from the estimate. Notifications for cost change follow whatever the backend sends.

24.5 FLOW 7 — Admin order (pending → fulfillment)

`/admin/orders` (Pending Review tab) → `/admin/orders/:reference` → review items, customer snapshot, stock context → Accept (`POST .../accept`, `expectedVersion`) or Reject (`reason`, `expectedVersion`; confirm dialog) → on accept the backend reserves stock for all lines atomically (UI shows result only; `INSUFFICIENT_STOCK` handled as above) → status becomes `accepted` → digital: `awaiting_payment`; COD: `customer_confirmation_required` → after payment confirmed / customer confirmed → Admin sets `preparing` through `POST .../status { targetStatus, expectedVersion, reason? }` → Pickup: `ready_for_pickup` → `picked_up` → `completed`; Delivery: choose provider and cost (`.../shipping`), `shipped` → `out_for_delivery` → `delivered` → `completed`. The frontend only offers the next statuses that match the documented lifecycle, but the backend's transition map is authoritative: an unexpected `ORDER_STATE_CONFLICT` reloads the record.

25. Payment Proof Workflow

25.1 When it applies (CONFIRMED)

For InstaPay and approved e-wallet methods only. Not for COD. The customer transfers outside the platform, uploads one or more screenshots, and never enters a transaction ID. Proof is reviewed manually by Admin after the Admin has accepted the order (IA: "Proof is reviewed manually after Admin accepts the order"; DB plan: proof submission is created after the order is accepted and payment is awaiting).

25.2 Timing: what exactly happens at checkout (CONTRACT TO VERIFY; FB-04)

The sources are consistent that the checkout *flow* contains a proof step (IA §6 sequence: "Payment proof for InstaPay/e-wallet ... upload one or more screenshots", and `POST /orders` "proof requirements satisfied where proof is submitted", failure `PAYMENT_PROOF_REQUIRED`) but also that the DB plan creates the proof submission *after acceptance* via `POST /orders/:reference/payment-proofs` . The sources do not say how a screenshot chosen at checkout reaches the backend before the order has a reference. Two readings exist (upload metadata with `POST /orders` ; or upload after the order exists). Do not choose silently. Safe behavior until the contract is confirmed:

1. The checkout payment-proof step is built exactly as the IA shows (select, validate, preview).
2. At submit, send whatever the `POST /orders` contract accepts for proof metadata. If the contract takes none, upload happens on the order page immediately after creation (the "Awaiting Payment / upload proof" state), and the confirmation page prompts it.
3. In both cases the order lands in `pending_review` ; the proof is reviewed by Admin only after acceptance; the UI says so.

25.3 Customer flow (FLOW 3)

Checkout (digital method) → proof step → submit order → `pending_review` → Admin accepts → `awaiting_payment` → customer uploads/confirms proof → payment `under_review` (order `payment_verification`) → Admin Confirm / Reject / Request New Proof → customer notified (in-app, email as configured) → Confirmed: `payment_confirmed` → `preparing` ; Rejected/New proof: `awaiting_new_proof` → customer re-uploads → `payment_verification` .

25.4 Upload mechanics (§29): `POST .../upload-config` → direct Cloudinary upload → `POST .../payment-proofs` with returned metadata (+ optional note + idempotency key).

25.5 Admin flow (FLOW 8)

`/admin/payments` queue (statuses `under_review` , `new_proof_requested` , oldest first) → payment detail: order context (reference, amount due, method snapshot), all proof submissions with history (submission number, files, status, reviewer note) → `ProofViewer` : requests a short-lived signed URL on demand per file (endpoint ◇), renders in an `<img>` with `referrerpolicy="no-referrer"` , no caching of the URL beyond the view, no copy/share affordance, URL discarded on close or after expiry (re-request on error) → actions Confirm, Reject (reason), Request New Proof (note), each with `expectedVersion` and confirm dialog for Confirm/Reject → result updates payment + order status; audit is server-side → customer notified.

25.6 States on the customer side

Payment status	UI
<code>not_submitted</code>	instructions + uploader (if order accepted)
<code>proof_uploaded</code> / <code>under_review</code>	"Payment proof is under review" + submitted thumbnails (only if backend exposes customer-viewable proof)

Payment status	UI
confirmed	"Payment accepted"
rejected / new_proof_requested	reason (if captured) + uploader enabled

Never say "paid" while under review. Never present payment details for COD.

26. Pre-order Workflow (FLOW 5)

Eligibility (CONFIRMED): only when the product/variant is out of stock and Admin marked it pre-order-eligible; it is an alternate action on the product page, not a second catalog.

1. Product detail shows Pre-order instead of Add to Cart; copy states no availability date is promised unless the library entered one.
2. `/products/:slug/pre-order` : variant (if applicable), quantity, contact (guest or customer per backend contract) → `POST /products/:slug/pre-orders` → success with reference and status, "Admin review".
3. Registered customers see it in `/account/pre-orders` (+ detail with `Timeline(preorder)` and notifications). Guest access to a pre-order record is not specified (FB-10): do not build a guest tracking route; show the reference and "sign in to follow" if the backend returns no guest token.
4. Admin: `/admin/pre-orders` → accept (optionally an expected-availability date) or reject; accepted → customer pays (payment availability after acceptance: the customer-side payment action and whether it reuses the proof flow are CONTRACT TO VERIFY); product becomes available → Admin marks available → the backend moves paid pre-orders into standard fulfillment from `preparing`.
5. The UI never shows stock reservation for a pre-order, and never promises price-honoring or allocation order (OD-10, OD-11). The captured price is shown as "price at the time of your request" (a neutral label) and nothing about whether it will be honored.

Statuses: `requested`, `admin_review`, `accepted`, `payment_pending`, `payment_verification`, `confirmed`, `available`, `rejected`, `fulfilled`, `cancelled`.

27. Service / Quotation Workflow (FLOW 6)

```

/services → /services/:slug → /services/:slug/request (submit)
→ [library review] → quotation sent → /account/services/:reference/quotation
→ Accept → payment → processing → completed | Reject → closed (not proceeding)

```

Step	Frontend behavior
Submit	<code>POST /services/:slug/requests</code> with dynamic fields + description + contact. Result: reference + status (<code>submitted</code> / <code>admin_review</code>). No file upload; explain WhatsApp/Telegram exchange.
Review	Request detail shows <code>Timeline(service)</code> ; "Waiting for the library quotation" copy; no price; WhatsApp CTA with the service reference
Quotation sent	Notification; detail shows <code>QuotationCard</code> : request context, quoted price (<code>amountMinor</code>), Accept / Reject. Only a <code>sent</code> quotation is visible to the customer. Validity/expiry: none shown (no policy; backend <code>expired</code> only if later approved)

Step	Frontend behavior
Accept	<code>POST /service-requests/:reference/quotation/accept { quoteVersion }</code> → request <code>awaiting_payment</code> , payment created. Payment UI becomes available only after acceptance; it reuses the payment panel and proof uploader (the sources mark "reuse the product payment/proof model for accepted quotations" as a PRODUCT RECOMMENDATION and COD applicability as OD-14 / FB-08); render the methods the backend returns for that payment; show COD only if the backend lists it
Reject	<code>POST .../reject { note?, quoteVersion }</code> → request closed (the auto-close behavior is a PRODUCT RECOMMENDATION ; the UI shows whatever status the backend returns, <code>closed_not_proceeding</code>); no payment
Version conflict	<code>409</code> → reload; if a newer quote exists show it
Payment → processing → completed	Statuses follow backend; no turnaround promise (OD-13)
Admin	<code>/admin/services/requests/:reference</code> : review fields, clarify via WhatsApp (contact customer), enter quote amount (EGP input → minor units) + note, send; track acceptance; progress statuses; payment verification uses the same payment-review component as products

Guest limitation: FB-09.

28. Returns / Refunds (FLOW 9)

- Entry: only from a `completed` order and only for items the backend marks eligible. If the backend flags an item non-returnable, the UI shows a clear ineligible reason and no request form for it. Periods, category rules, evidence rules, and refund SLA are open (OD-15, OD-16 / FB-07): the UI shows neutral, configurable wording and never states a return window. The only confirmed policy case is wrong/damaged item.
- Request: `POST /orders/:reference/returns` with items (`orderId`, `quantity`, `reason`) and evidence metadata only if the backend accepts it. (If evidence involves file upload, the upload mechanism is not specified: **CONTRACT TO VERIFY**; do not reuse the payment-proof folder.) Success → `return_requested` with reference, then `ReturnDetailPage`.
- Customer view: `Timeline(return)`: `return_requested` → `return_review` → `return_approved` → `refund_initiated` → `refund_completed` or `return_rejected`; refund record displayed (amount, method, time) when it exists, shown as a separate record from the original payment and proofs. There is no customer refund action.
- Admin: `/admin/returns` queue → review items/reason/evidence → Approve (`POST .../approve`, note, `expectedVersion`) → refund `initiated`; Reject (reason) → terminal. Refund record page `/admin/refunds/:refundId`: Complete with amount, method, timestamp, internal reference → `POST .../complete`. Refunds are manual (InstaPay/e-wallet by the library); the UI never implies an automated refund, never invents a refund duration. Repeat completion is idempotent: use an idempotency key; `409` shows the current state.
- Notifications for return/refund updates arrive through the same notification pipeline.

29. File Uploads

Scope (CONFIRMED): payment proof screenshots (and product images in Admin, subject to a contract check). Service attachments are not supported anywhere. Return evidence: only if the backend defines it.

29.1 PaymentProofUploader

Concern	Rule
Input	Accessible drop zone + button (<code><input type="file" accept="image/png,image/jpeg,image/webp" multiple></code>); on mobile the camera/gallery picker via the native input. Keyboard operable; label and instructions visible.
Allowed types	PNG, JPEG, WebP (backend default; may be narrower via ops config: read allowed types and max size from the <code>upload-config</code> response when it provides them (CONTRACT TO VERIFY); otherwise client uses conservative defaults and relies on backend rejection).
Size	The exact limit is configurable on the backend (not specified). Do not hard-code a number as policy: take it from <code>upload-config</code> or config; until known, validate type only and treat a backend/Cloudinary size rejection as a normal error with a clear message.
Count	One or more allowed. A maximum count is not specified; do not invent one.
Validation	Type by MIME and extension; reject empty files; show per-file error; do not trust client validation (backend re-validates).
Preview	<code>URL.createObjectURL</code> thumbnails, revoked on remove/destroy. No upload of the full image to any service other than Cloudinary.
Flow	1) <code>POST .../payment-proof/upload-config</code> with file metadata → signed constrained config. 2) Upload directly to Cloudinary with <code>XMLHttpRequest / HttpClient</code> <code>reportProgress</code> (no auth interceptor header; the interceptor must not attach the bearer token to Cloudinary). 3) <code>POST .../payment-proofs</code> with returned metadata (public id, resource type, format, bytes, width/height), optional note, idempotency key.
Progress & cancel	Per-file progress bar; cancel aborts the in-flight upload (<code>unsubscribe/ abort</code>); canceled files can be re-added. Cancelling does not create a proof record because step 3 never ran.
States	idle → selected → validating → uploading (progress) → uploaded (pending submit) → submitting → success; error at any step with Retry (re-run from the failed step, same idempotency key for step 3).
Retry	Retry reuses the signed config only if not expired; otherwise requests a new one. Duplicate submissions are prevented by key + disabling.
Success	Announce via live region; show "Awaiting payment verification" (not "Paid").
Security	The signed URL/signature is never logged or persisted. Never display payment-proof images through public unsigned URLs. EXIF/privacy: no client-side image editing is required.
Admin viewing	See §25.5.
Failure of Cloudinary or backend	State kept; message explains what was and was not saved ("your file was not submitted"); retry available; WhatsApp link.

29.2 Product images (Admin)

The backend plan says product media fields store Cloudinary metadata and the backend validates image metadata (§21.3, §37.1). Whether the client uses the same signed direct-upload flow with a different context is a CONTRACT TO VERIFY. If so, reuse the same `FileDropzone` /upload service with a different context parameter; otherwise defer the image manager until the endpoint is confirmed. Missing images use the calm placeholder; product publication is not blocked solely by missing description.

30. SEO

Requirement (CONFIRMED, PRD §15): product and category pages must be indexable and structured to communicate specialization, product type, and location; copy must not stuff "مكتبة الأزهرى"; social sharing of product/category pages should be supported (Facebook/TikTok conversion). PRODUCT RECOMMENDATION: expose title, description, category, availability for search and social previews. The PRD explicitly says it does not define technical SEO.

30.1 Decision FB-02 — SSR / prerender / SPA (OPEN DECISION — DO NOT INVENT)

The sources do not say SSR is required. They do require crawlable pages and social previews. Facts for the owner/tech lead:

- Social crawlers (Facebook, TikTok, WhatsApp link previews) generally do not execute JavaScript. A client-rendered SPA cannot reliably deliver per-product Open Graph tags.
- Backend deploys to Hostinger Node.js with a same-origin strategy; Angular SSR needs a Node process too, so the hosting/proxy design changes if SSR is chosen.
- Options: (a) SPA only (simplest; weak previews), (b) Angular SSR for public routes (/ , /shop* , /categories/* , /products/:slug , /services* , /contact) with private routes client-only, (c) prerender static routes + a lightweight meta-injection for dynamic product pages.

Safe interim behavior: implement everything SSR-safe (§5.3) and route all meta through `SeoService`, so choosing (b) or (c) later does not require refactoring. Treat share-preview quality as unverified until FB-02 is closed.

30.2 `SeoService` (public pages only)

Item	Rule
Title	From route <code>data.titleKey</code> or a resolver (product/category/service name); suffixed by the brand name once; no repeated brand stuffing
Description	Product: concise from authored description if present, otherwise composed from category + populated metadata (author/grade/stage) with no invented claims; category/service: from content or a translated default
Canonical	Absolute URL of the route without volatile params (<code>cursor</code> , <code>page</code> , <code>sort</code> if non-canonical)
Robots	<code>index, follow</code> for Home, Shop landing, All products, Category, Product, Services pages, Contact. <code>noindex</code> for <code>/search</code> , cart, checkout, orders, account, auth, admin, pre-order/request forms (FB-14 asks the owner to confirm search-page indexing)
Open Graph / Twitter	<code>og:title</code> , <code>og:description</code> , <code>og:type</code> (<code>product</code> for product pages), <code>og:url</code> , <code>og:image</code> = the product's full cover (contained, no overlay) with fallback brand image; <code>og:locale</code> . Only if FB-02 makes them effective
Structured data	JSON-LD <code>Product</code> (name, image, description if present, sku/isbn if present, offers: price in EGP from minor units, availability mapped from <code>in_stock/out_of_stock/pre_order_eligible</code>) and <code>BreadcrumbList</code> . No <code>aggregateRating</code> or review markup (no reviews exist). <code>Organization/Store</code> markup only with confirmed address/contact (OD-01, Q-02): omit until confirmed
Sitemap/robots.txt	Generation is a deployment/backend or build task not defined by the sources (PRD calls these implementation detail). Needs an owner (FB-02); the frontend must not hard-code a product list

Item	Rule
Language	<code><html lang></code> per active locale; no hreflang (no locale in URLs; FB-14)
Content tone	Natural Arabic, no keyword repetition; headings semantic
Private pages	Never expose data in meta; <code>noindex</code> set before data loads

31. Accessibility

Target (recommendation, OD-19): WCAG 2.2 AA; the formal level is not confirmed. Build to AA now.

Area	Implementation
Semantics	Landmarks (<code>header</code> , <code>nav</code> , <code>main</code> , <code>footer</code>), logical heading order, <code><button></code> vs <code><a></code> correctness, tables with <code>scope</code> and captions
Keyboard	Everything operable; logical tab order that follows reading order in RTL; no keyboard traps except intentional modal traps; <code>Esc</code> closes dialogs/drawers/menus; skip link
Focus	Visible 2px ring (primary green <code>#17624C</code> focus ring token + offset); focus restored to the trigger on dialog/drawer close (CDK <code>FocusTrap</code> / <code>restoreFocus</code>); focus moved to <code><h1></code> on route change; focus moves to the first error only on submit
Forms	Visible labels (placeholder is not a label), helper/error via <code>aria-describedby</code> , <code>aria-invalid</code> , required marked textually, input modes, <code>autocomplete</code> tokens (<code>name</code> , <code>tel</code> , <code>email</code> , <code>street-address</code> , <code>family</code> , <code>new-password</code> / <code>current-password</code>)
Errors/async	<code>aria-live="polite"</code> for results/loading completion; <code>role="alert"</code> for blocking failures; skeletons <code>aria-busy</code> , not announced as content
Dialogs/drawers	CDK dialog with <code>role="dialog"</code> / <code>alertdialog</code> for confirmations, labelled title, focus trap, scroll lock; mobile filter sheet is modal with the same semantics; the nav drawer uses navigation semantics when non-modal
Menus/dropdowns	CDK menu/listbox patterns, arrow-key navigation, typeahead optional
Status	Never color alone: text + icon; <code>StatusBadge</code> has accessible text
Icon-only buttons	Localized accessible names; tooltips on desktop; WhatsApp CTAs have distinct names per context ("اسأل عن هذا الكتاب على واتساب")
Contrast	Tokens verified AA: green-700 on white/ivory; gold-500 never as small text on white (use gold-700); check reversed text on green for Arabic and Latin separately; disabled content remains legible
Touch targets	≥ 44×44 customer; ≥ 40×40 dense admin icon actions with spacing
Motion	Respect <code>prefers-reduced-motion</code> (no transforms, shimmer, scale); never delay feedback for animation
RTL	Screen-reader order = DOM order (no visual-only reordering via CSS <code>order</code> for meaningful sequence); direction icons flip; LTR islands isolated; <code>lang</code> attributes on foreign fragments; no punctuation added for visual alignment

Area	Implementation
Images	Localized meaningful alt (product name, not "image of book"); decorative motifs <code>aria-hidden</code> ; gallery controls labelled with position ("2 of 4")
Realtime	New notifications announced via a throttled polite live region
Tables (admin)	Real <code><table></code> semantics on desktop; card fallback keeps label/value pairs; sortable headers expose <code>aria-sort</code>
Testing	axe in unit/e2e for key pages; manual keyboard + screen reader (NVDA/VoiceOver, Arabic) pass for catalog, product, checkout, order tracking, admin order detail, admin payment review

32. Responsive Design

Design System breakpoints are the single source (`BreakpointObserver` wrappers expose signals `isMobile`, `isTablet`, `isDesktop`; CSS uses the same tokens). Never shrink desktop UI until labels are unreadable.

Layout	Mobile (xs-sm, <768)	Tablet (md, 768-1023)	Desktop (lg+, ≥1024)
Storefront header	menu button, logo, search, cart; account inside menu; persistent WhatsApp that never covers primary action	expanded search, more links visible	full nav: logo, Shop (leading), Services, Search, Account, Cart, WhatsApp
Navigation	slide-in drawer from the inline-end/start per direction; Shop first	drawer or inline	inline
Catalog grid	1-2 columns (2 only when title/price/action readable); filters in bottom sheet; active filters visible above grid	2-3 columns; filter drawer or 2-col panel	4 columns (5 only after visual QA); inline filters/sidebar
Product cards	stacked image-over-text; 44px action	same	same with border hover
Product detail	stacked: gallery → info → action; sticky bottom primary action	gallery/info split where viable	two-column editorial layout; WhatsApp near purchase action
Cart	stacked rows; sticky total + Proceed	roomy rows	rows + side summary
Checkout	single column; collapsible/sticky bottom summary with total + next step visible; changes auto-expand	single or split	~2/3 form + ~1/3 sticky summary
Forms	one column, full-width controls, visible labels, ≥44px height	one-two columns	max readable width (<code>container-content</code>)
Dialogs	bottom sheets for selection/filter; centered dialogs for confirmations	either	centered dialog or drawer per task
Order timeline	vertical, labelled	vertical or compact horizontal if labels fit	horizontal summary may accompany vertical detail
Account	stacked section menu; current state first	compact sidebar	compact sidebar
Services	single column; one primary action per screen	same	reading-width container

Layout	Mobile (xs-sm, <768)	Tablet (md, 768-1023)	Desktop (lg+, ≥1024)
Admin shell	drawer nav + compact top bar	compact sidebar	persistent sidebar
Admin tables	priority-card fallback or horizontally scrollable region with preserved headers; actions never hover-only	table with priority columns	full table + filters + bulk actions
Admin forms	single column grouped by task	two columns	two columns, sticky action bar
Notification UI	bell opens a full-width sheet	popover	popover

Container widths and grid: reading 720, content 1120, wide 1280, max 1440; at 1920 cap the container. Validate in Arabic and English at 375, 480, 768, 1024, 1280, 1440, 1920, including long Arabic titles and mixed Arabic/Latin metadata.

33. Error Handling

33.1 Pipeline

`HttpErrorResponse` → `errorInterceptor` → `ApiError` (`code` , `status` , `safeMessageKey` , `fields?` , `requestId` , `retryable`) → store/component decides presentation using `ErrorPresenter` (one mapping table, `core/errors`). A `GlobalErrorHandler` catches unexpected client exceptions, logs a redacted report (never tokens/PII), and shows a generic recoverable state. Do not invent codes: unknown `code` → generic message for its HTTP status class.

33.2 Status → behavior

HTTP / code	Meaning	UI behavior
400 <code>VALIDATION_ERROR</code>	invalid input	map <code>fields</code> to controls; banner for unmapped; keep input
401 <code>AUTH_REQUIRED</code>	missing/invalid token	refresh-and-retry once (§10.4); on failure → login with <code>returnUrl</code> (protected route)
401 <code>AUTH_INVALID_CREDENTIALS</code>	login failed	one generic message, no hint which field
401/410 <code>AUTH_TOKEN_INVALID</code>	verify/reset token bad/expired	state explaining + path to request a new link
403 <code>FORBIDDEN</code>	permission/ownership denied	<code>/403</code> for pages; in-place "no permission" for actions; no data leakage
404 <code>NOT_FOUND</code>	safe not found	<code>/404</code> or in-place not-found with Shop/Search/Home
409 <code>RESOURCE_CONFLICT</code> / <code>ORDER_STATE_CONFLICT</code>	version/state conflict	reload record, explain "this record changed", keep user's intent visible
409 <code>INSUFFICIENT_STOCK</code>	cannot reserve/add	specific message; admin: stays in Pending Review; customer: update cart
409 <code>IDEMPOTENCY_KEY_REUSED</code>	same key, different payload	new key; ask to re-review
409/422 <code>PRICE_CHANGED</code> , <code>AVAILABILITY_CHANGED</code>	checkout revalidation	return to Review with change list
422 <code>PAYMENT_PROOF_REQUIRED</code>	proof missing	go to proof step
422 <code>SHIPPING_CONFIGURATION_UNAVAILABLE</code>	no approved rule	neutral message + pickup/WhatsApp (OD-03/06)

HTTP / code	Meaning	UI behavior
422 <code>PAYMENT_METHOD_UNAVAILABLE</code>	method disabled	back to Payment step, refresh methods
422 <code>BUSINESS_RULE_VIOLATION</code>	confirmed rule blocks action	explain with a generic translated message; if a documented <code>details</code> key exists, map it; otherwise generic
429 <code>RATE_LIMITED</code>	throttle	"try again shortly", disable submit briefly; honor retry hint if present; for guest order access use the neutral state
500 <code>INTERNAL_ERROR</code>	unexpected	generic error state, retry, request ID in disclosure, WhatsApp
503 <code>DEPENDENCY_UNAVAILABLE</code>	external adapter down	"temporarily unavailable"; retry; for uploads see §29
Network failure / offline	no response	non-blocking offline hint; for mutations say <i>whether anything was saved</i> (it was not, unless a retry with the same key proves otherwise); retry; preserve inputs
Timeout	slow	same as network; for create-order retry with the same idempotency key

33.3 Message rules

Localized, short, actionable. Never expose stack traces, Mongoose/Mongo wording, internal ids, or whether an account exists. Never claim an order failed solely because a real-time message failed: reload the authoritative record.

34. Loading / Empty / Error States

Standardized components: `Skeleton*` (card grid, product detail, order detail, table rows, timeline), `EmptyState`, `ErrorState`, inline `Spinner` (button-local only), `Alert`.

State	Pattern	Rules
Initial loading	skeleton matching final structure	reserved dimensions, no layout jump; <code>aria-busy</code>
Refreshing	keep content, subtle top progress or inline indicator	do not blank the page
Empty	explanation + next useful route	Cart → Shop/Categories; No results → clear/refine; No orders → Shop; No services → Service Catalog; No notifications → neutral; No payment history → Orders; Admin empty queue → "queue is clear" without celebrating
Partial data	render loaded sections; failed section shows local <code>ErrorState</code>	home modules, account overview, order detail panels
Validation error	field errors + summary on long forms	
API error	<code>ErrorState</code> (what failed, whether saved, retry, safe navigation, optional WhatsApp, request-ID disclosure)	
Permission denied	<code>/403</code> or in-place	
Not found	<code>/404</code> or in-place	

State	Pattern	Rules
Success	precise: order submitted → "Pending Review"; proof uploaded → "Awaiting payment verification"; service request submitted → "Admin Review"; return submitted → "Return Requested"	never "confirmed/paid/refunded" prematurely
Mutation pending	disabled + spinner, label kept, width stable	
Mutation success	inline result + toast (non-critical)	critical results are persistent
Mutation failure	inline + field mapping; keep input	

Avoid spinners on every request: only for button-local actions; page-level uses skeletons.

35. Security

Frontend security hardens the client; the backend remains the authority for every item marked ☞.

Topic	Practice
Token handling	Access token in memory only; refresh cookie HttpOnly (unreadable); no tokens in URLs (except the one-time guest/verify/reset tokens, stripped immediately), logs, analytics, or WhatsApp text
Token leakage	Attach bearer only to <code>apiBaseUrl</code> ; never to Cloudinary or other origins; <code>Referrer-Policy</code> strict; no tokens in <code>console</code>
Secrets	None in environment files or bundles. Public identifiers only
XSS	Angular's default escaping; no <code>innerHTML</code> / <code>bypassSecurityTrust*</code> for any backend/Admin-authored text; descriptions and CMS content render as text (or a tiny whitelisted formatting subset if later approved); no <code>eval</code> ; CSP with no <code>unsafe-inline</code> for scripts is a deployment target
URLs	Any backend-supplied URL (<code>actionUrl</code> , WhatsApp link, social links, product media) validated: internal routes via an allowlist (<code>/^\/[^\//]</code> and known prefixes) for <code>actionUrl</code> ; external links only <code>https:</code> and only to allowlisted hosts (WhatsApp, Telegram, Facebook, TikTok, Cloudinary); <code>rel="noopener noreferrer"</code>
Open redirects	<code>returnUrl</code> must be an internal path; otherwise ignore
File upload	Client type/size checks are UX only ☞ backend + Cloudinary signed policy enforce; never render user-uploaded files as HTML; previews through object URLs
Payment proof rendering	Only through short-lived signed URLs for authorized roles ☞; render in <code></code> with <code>no-referrer</code> ; no persistence, no sharing controls; discard on close
Sensitive data rendering	Show only what the API returns; mask nothing client-side that the backend should have redacted; no PII in telemetry; guests never see other orders
Permission checks	UI hides/disables for UX ☞ backend enforces permission + ownership; handle 403 gracefully
Secure logout	Call logout; always clear stores, caches, socket, guest tokens, drafts; navigate away from protected routes; <code>Clear-Site-Data</code> is a backend option, not assumed
Session expiry	Single notice, no loops (§10.4); protected routes redirect with <code>returnUrl</code>

Topic	Practice
CSRF	Same-origin deployment: refresh cookie <code>SameSite</code> set by backend ☹️. If frontend is hosted cross-site later, backend adds CSRF tokens (backend §50); the frontend would then need to send that token on refresh/logout; do not deploy cross-site without that decision
Clickjacking/headers	<code>frame-ancestors</code> /CSP/ <code>X-Content-Type-Options</code> served by hosting/backend ☹️
Dependencies	Lockfile, audit on CI, minimal dependency set
Idempotency	Keys are random UUIDs, not derived from secrets
Guest token	In sessionStorage/memory only; never logged; FB-16
Rate limiting	Handle 429 gracefully; backend enforces ☹️

36. Performance

- Lazy loading: every top-level group and every admin module is a lazy route; `canMatch` prevents loading admin for others. Dictionaries scoped and lazy.
- Preloading: network-aware; preload catalog/product/cart after first paint; never admin for customers.
- Bundle discipline: no heavy libraries (no UI suite, no chart library by default, no date/money libs). Lucide icons imported individually into the registry. Budgets in `angular.json` (set initial JS budget at kickoff and fail CI on regression; exact numbers are a team decision, not a source requirement).
- Images: product covers use `loading="lazy"` below the fold, `decoding="async"`, intrinsic `width/height` /aspect-ratio box (2:3) to avoid CLS, Cloudinary transformation URLs for responsive `srcset` / `sizes` (format/size parameters per the backend's media policy: CONTRACT TO VERIFY), the LCP hero/first product image is `fetchpriority="high"`. Use `NgOptimizedImage` where the loader can be configured for Cloudinary; otherwise a small `ProductImage` component.
- Fonts: self-hosted, subset, `font-display: swap`, preload only the primary Arabic and Latin weights.
- Pagination: cursor "Load more"; virtual scrolling only if admin lists prove large (CDK scrolling), not by default.
- API caching: only the safe list in §11.1; `shareReplay` with `refCount` for in-flight dedupe of identical GETs.
- Change detection: OnPush + signals everywhere; `@for` with `track`; avoid function calls in templates (use `computed`).
- Subscriptions: prefer `toSignal`, `async`, and `takeUntilDestroyed`; no manual subscribe without teardown. Realtime listeners registered once.
- Rendering: SSR/hydration if FB-02 selects it; otherwise skeletons to protect perceived performance.
- No unnecessary infrastructure: no service worker/PWA, no analytics SDKs, no state libraries unless a later decision justifies them.
- Targets: formal performance targets are OPEN (OD-19); measure Core Web Vitals on the catalog and product pages and record a baseline in Phase 13 rather than inventing numbers.

37. Testing Strategy

Level	Tools	What to test	What not to over-test
Unit: pure logic	Jest/Jasmine	mappers (DTO→domain), money conversion, status presentation map (every documented status has label/tone/icon), query serializers/parsers, idempotency key lifecycle, error mapper, URL/redirect validators, guest token store	trivial getters, framework behavior
Service/API	HttpTestingController	each <code>*Api</code> builds the right URL/method/params, unwraps the envelope, maps errors, sends <code>expectedVersion</code> /idempotency where required	backend business logic (that's backend tests)
Interceptors	HTTP testing	auth header only on API origin; single shared refresh for N concurrent 401s; no retry loop; excluded endpoints; refresh failure → expired once; error normalization; idempotency attachment	
Guards	router testing	<code>authGuard</code> , <code>guestOnly</code> , <code>adminMatch</code> (bundle not loaded), <code>checkoutStepGuard</code> , <code>preOrderEligibleGuard</code> , <code>returnUrl</code> safety	
Stores	unit	<code>CartStore</code> (add/update/remove/conflicts/stale 409), <code>AuthStore</code> transitions, <code>NotificationStore</code> dedupe/revalidate, <code>CatalogStore</code> URL sync	
Forms	component/unit	validators, server-error mapping to controls, dynamic service form builds only active fields, no file control in service request, proof uploader validation	pixel details
Components	Angular testing + axe	ProductCard (states: in stock/out/pre-order/missing image/missing metadata), StatusBadge, PaymentProofUploader, WhatsAppCta (hidden without config; no tokens in URL), DataTable responsive fallback, ConfirmDialog focus trap/restore	every presentational variant snapshot
RTL/LTR	component + visual	key components render correctly in both directions; direction icons flip; LTR islands	
Integration (frontend + mocked API)	Playwright with a mock server (e.g., MSW or Playwright route mocking)	critical workflows below with realistic envelopes and error codes	

Level	Tools	What to test	What not to over-test
E2E (staging backend)	Playwright	smoke: guest checkout COD, customer checkout with proof, admin accept, admin payment confirm, service request → quote → accept	exhaustive permutations
Accessibility	axe + manual	per §31	
Visual regression	optional Playwright screenshots at 375/768/1280 in RTL for core pages	against the <code>design-website</code> references once audited	

Critical workflow suites (priority order): (1) authentication: login, restore, refresh-once, logout, expired session; (2) catalog: URL filters round-trip, empty/no-result, availability states; (3) cart: guest add/update/remove, stale version, merge conflict dialog; (4) checkout: guest COD, registered digital, shipping not serviceable, price changed, idempotent retry after network failure, missing proof blocked; (5) payment proof: validation, upload failure/retry, re-upload after rejection; (6) orders: status rendering for every status, cancel only in pending review, COD confirm; (7) services: no cart interaction, quote accept/reject, closed state, no price before quote; (8) admin authorization: non-admin cannot load admin, 403 handling, accept conflict, reject proof; (9) returns/refund display; (10) notifications: socket event → revalidate; socket down → focus refresh shows persisted state.

Coverage target: an exact percentage is a team decision. Gate CI on: all suites above green, status-map completeness test, boundary-lint pass, axe pass on key pages.

38. Implementation Phases

Dependency-aware. Each phase lists goals, tasks, affected areas, dependencies, API dependencies, completion criteria, and required tests. Phases 4–8 can partially overlap once Phase 3 is done; Admin (9–12) can start in parallel with storefront once Phases 0–3 are done and a second developer is available.

Phase 0 — Foundation and decisions

- Goals: a buildable, linted, empty Angular workspace and the blocking decisions recorded.
- Tasks: create the Angular workspace (standalone, strict TS, SCSS); pin Angular version; ESLint + boundaries + Stylelint (logical properties) + Prettier; CI (lint, test, build); environment files (non-secret); folder skeleton from §7; `PlatformService / SafeStorage`; app bootstrap with `lang="ar" dir="rtl"`; obtain the OpenAPI/Postman collection and resolve every CONTRACT TO VERIFY item that gates Phase 2–5 (write answers into `docs/api-contract-notes.md`); audit `design-website/` (Gap A); confirm FB-01 (styling), FB-02 (SSR), FB-12 (palette/fonts/logo assets).
- Dependencies: none. API deps: documentation only.
- Done when: empty app builds and lints in CI; contract notes and design audit exist; FB-01/02 are decided or have a stated interim.
- Tests: CI smoke.

Phase 1 — Design system and shared primitives

- Goals: tokens + primitives matching the audited mockups.
- Tasks: token SCSS (§14); fonts; icon registry; `Button`, `Input` family, `Badge`, `StatusBadge` + `StatusPresentation`, `Alert`, `Card`, `Skeleton`, `EmptyState`, `ErrorState`, `Tabs`, `Breadcrumbs`, `Tooltip`, `Dialog/Drawer/BottomSheet/Toast` (CDK), `FormField`, `QuantityStepper`, `PriceDisplay`, `AvailabilityBadge`, `LtrText`, `WhatsAppCta`, `Timeline`, `DataTable`; a living style page (non-routed or dev-only route) showing every component in AR/EN × all states.

- Dependencies: Phase 0. API deps: none.
- Done when: each primitive matches its mockup in AR/RTL and EN/LTR at 375/768/1280, passes axe, and documents states.
- Tests: component tests, axe, status-map completeness.

Phase 2 — Core infrastructure and API layer

- Goals: all plumbing needed by features.
- Tasks: `ApiClient`, envelope/pagination types, DTOs and mappers, `ApiError` and error code map, interceptors (request-id, locale, auth, idempotency, error), `AppConfigStore` + `ConfigApi`, i18n/direction/locale services and dictionaries skeleton, `SeoService`, `AnalyticsService` (no-op), storage/guest token store, `MoneyService/MoneyPipe`, date pipes, `RealtimeService` skeleton (no connection until Phase 8), global error handler, `ConfirmDialog` wiring, route typing and `NetworkAwarePreloading`.
- Dependencies: Phase 0–1. API deps: envelope/errors; `GET /categories` for a smoke check.
- Done when: a throwaway page calls a real endpoint and renders loading/error/empty through shared states; refresh-loop test passes.
- Tests: interceptor, mapper, error mapper, money, query serializer unit tests.

Phase 3 — Authentication

- Goals: complete session handling and auth screens.
- Tasks: `AuthApi`, `AuthStore`, token holder, `restore()` initializer, guards, Login/Register/Verify/Forgot/Reset pages, logout, `returnUrl` safety, unverified banner support (FB-05 safe behavior).
- Dependencies: Phase 2. API deps: `/auth/*`, `GET /me`.
- Done when: register→verify→login→refresh→logout works end to end; concurrent 401 triggers one refresh; expired session behaves per §10.4.
- Tests: auth interceptor/guard/store suites; form tests; E2E smoke.

Phase 4 — Storefront and catalog

- Goals: discovery, search, product detail.
- Tasks: `StorefrontShell` (header/footer/mobile nav), Home, Shop, All products, Category, Search, `CatalogStore` + URL sync, filters (drawer on mobile), `ProductCard/Grid/Gallery/VariantSelector`, Product detail (states: in stock, out + eligible, out + not eligible, variants, not found), Contact, 404/403, SEO meta per §30.
- Dependencies: Phases 1–3 (auth only for header state). API deps: `/categories`, `/products`, `/products/:slug`, `/search`, `/content/home`, public config (contact/WhatsApp).
- Done when: every catalog state works in AR/EN at all breakpoints; filters round-trip through the URL; no reviews/ratings anywhere.
- Tests: catalog store/URL tests; ProductCard/detail state tests; a11y.

Phase 5 — Cart and checkout

- Goals: guest and registered purchase.
- Tasks: `CartStore` / `CartApi`, header cart mini, Cart page with change sets and merge dialog (per verified contract), `CheckoutStore`, steps 1–6, shipping estimate, payment methods from config, order submit with idempotency, confirmation page, guest token store.
- Dependencies: Phases 3–4. API deps: `/cart*`, `/checkout/shipping-estimate`, `POST /orders`, payment-method config.

- Done when: guest COD and registered digital orders can be submitted; retry after network failure does not duplicate; price/availability change handled before payment.
- Tests: cart store, checkout flow integration, idempotency, error-code UI.

Phase 6 — Orders and customer dashboard

- Goals: post-purchase visibility and account.
- Tasks: `AccountShell`, overview, profile, addresses, orders list, Order detail/tracking (guest + customer), cancel, COD confirm, payments history, pre-order pages (customer side), return request/detail (customer side), shared `Timeline` states for every order status.
- Dependencies: Phases 3, 5. API deps: `/orders/*`, `/me`, `/addresses`, list endpoints (verified), `/pre-orders`, `/orders/:reference/returns`.
- Done when: every order status renders correctly; cancel only in pending review; guest tracking works with token.
- Tests: status rendering matrix; ownership/guest access; cancel conflict.

Phase 7 — Services and quotations (customer side)

- Goals: the separate service lifecycle.
- Tasks: services landing/catalog/detail, `DynamicServiceForm`, request submit, account service list/detail, `QuotationCard`, accept/reject, service payment panel (reusing the payment/proof panel when the backend supports it), copy for applications/transfers, WhatsApp/Telegram note.
- Dependencies: Phases 3, 6 (payment components). API deps: `/services*`, `/service-requests/*`.
- Done when: no cart interaction anywhere; no price before quote; no file control; empty `fields[]` renders safely.
- Tests: separation tests, dynamic form tests, quote decision tests.

Phase 8 — Notifications and Socket.IO

- Goals: persisted notifications and live delivery with fallback.
- Tasks: `NotificationsApi`, `NotificationStore`, bell + list page, `RealtimeService` (handshake auth, reconnect, rooms per verified contract, router, dedupe, cleanup), polling/focus fallback.
- Dependencies: Phases 3, 6. API deps: notification endpoints (verified), Socket.IO events.
- Done when: with socket blocked, everything still updates on focus/poll/reload; no duplicate listeners/events; logout disconnects.
- Tests: realtime router/dedupe, reconnect revalidation, fallback.

Phase 9 — Admin foundation

- Goals: secure admin shell and shared admin patterns.
- Tasks: `AdminShell`, `adminMatch`, permission-aware nav, `OperationalQueueTable`, detail layout, mutation helper (expectedVersion, 409/403/422 handling), confirm patterns, admin dictionary, dashboard, notifications page, role badge.
- Dependencies: Phases 1–3 and 8 (admin bell). API deps: `/me`, admin queue endpoints (verified).
- Done when: non-admin never downloads the admin bundle; forbidden handling consistent.
- Tests: guard/CanMatch tests, mutation helper, table responsive fallback.

Phase 10 — Admin commerce

- Goals: product, catalog, orders, inventory, shipping.

- Tasks: Orders queue/detail with accept/reject/status/shipping/COD reminder; Products list + editor + variants; Categories; Inventory adjust; Shipping queue.
- Dependencies: Phase 9. API deps: `/admin/orders*`, `/admin/products*`, `/admin/categories`, `/admin/inventory/*`, verified lists.
- Done when: the Flow 7 sequence works; stock errors shown correctly; no partial acceptance UI.
- Tests: accept conflict/insufficient stock; editor validation; money conversion.

Phase 11 — Admin payments, services, pre-orders, returns/refunds

- Tasks: Payment queue + `ProofViewer` (signed URLs) + confirm/reject/request-new-proof; Services queue/detail/catalog editor, quote form; Pre-orders; Returns queue/detail; Refund completion form.
- Dependencies: Phases 9–10, 6–7 for shared components. API deps: admin payments/services/pre-orders/returns/refunds endpoints.
- Done when: Flows 5, 6, 8, 9 work end to end from the Admin side; proof URLs never persisted.
- Tests: payment review, quote creation, refund completion idempotency.

Phase 12 — Reports, content, coupons, customers, audit, settings

- Tasks: Reports (approved dimensions only); Content modules; Coupons; Customers; Audit; Settings editors (typed per key).
- Dependencies: Phase 9. API deps: `/admin/reports/*`, `/admin/audit-logs`, `/admin/settings/*`, verified content/coupon/customer endpoints.
- Done when: no KPI targets/gauges; coupon fields limited to exposed ones; settings editors validate typed values.
- Tests: coupon unit conversion; settings forms.

Phase 13 — SEO, accessibility, performance

- Tasks: implement the chosen FB-02 approach; meta/JSON-LD; robots/noindex verification; full a11y audit (keyboard + screen reader, Arabic); performance pass (image/fonts/bundles, Core Web Vitals baseline); English dictionary completion for storefront; empty/error state audit across all pages.
- Dependencies: Phases 4–12. Done when: DoD (§39) holds for all pages and the open-decision interim behaviors are verified.

Phase 14 — Testing and production hardening

- Tasks: E2E suites on staging; visual regression versus audited mockups; security review (§35 checklist); CSP/headers coordination; deploy pipeline with the backend deployment strategy (same-origin on Hostinger or approved alternative); error monitoring (privacy-safe); release checklist; runbook.
- Done when: all critical workflow suites pass on staging; owner sign-off on the open-decision interim behaviors.

39. Definition of Done

A feature (page, component, flow) is done only when all applicable items are true:

1. Design: matches the audited mockup (layout, typography, spacing, tokens) or documented deviation approved.
2. Responsive: verified at 375/480/768/1024/1280 (and 1440/1920 for public pages).
3. RTL/LTR: correct in Arabic/RTL (primary) and English/LTR; logical properties; icon direction; LTR islands.

4. API integration: uses only verified endpoints through the API layer; no CONTRACT TO VERIFY left unresolved (or a ticket exists and a safe interim is implemented and documented).
 5. Typed models: DTO→domain mapper with tests; no `any`; unknown enum values handled.
 6. Loading, empty, error, success, mutation pending/success/failure, partial data, not found, forbidden: all designed and implemented.
 7. Permissions: route guard + UI gating where needed; handles backend 401/403; no authorization logic that only exists client-side.
 8. Validation: typed form, server error mapping, preserved input.
 9. Accessibility: keyboard complete, labels, focus management, live regions, axe clean, contrast, reduced motion.
 10. Localization: no hard-coded strings; AR complete, EN complete for public surfaces (admin EN may be partial per §13).
 11. Open decisions: no policy invented; configurable or pending behavior in place.
 12. Security: no `innerHTML`, no token leakage, URLs validated; proofs via signed URLs only.
 13. Tests: unit + component tests for logic/states; workflow test for critical paths; no flaky tests.
 14. SEO: meta for public pages; `noindex` for private pages.
 15. Performance: lazy loaded, OnPush, images sized, no unnecessary subscriptions.
 16. Analytics: only the supported events, no PII.
 17. Documentation: route/component noted in the traceability table; contract notes updated; Storybook-like style page entry for new shared components.
 18. Review: boundary lint, strict TS, CI green.
-

40. Traceability Matrix

Requirement groups use the ranges verified in the backend plan's traceability table (individual requirement texts were not exhaustively reviewed for this plan; where a developer needs the exact text, open the PRD section for that ID range). "Backend" lists the module and primary endpoints; ◇ endpoints need verification.

PRD requirement group	Frontend feature	Routes	Angular components / stores	API service	Backend module / endpoint	Test coverage
AUTH-001-012	Auth, profile, addresses, guest checkout	/login , /register , /verify-email , /forgot-password , /reset-password , /account/profile , /account/addresses	AuthStore , auth pages, ProfileForm , AddressForm , guards	AuthApi , MeApi , AddressesApi	auth/users/addresses: /auth/* , /me , ◇ /addresses	Auth interceptor/guard/store; form tests; E2E auth
CAT-001-009, VAR-001-004	Catalog, categories, variants	/shop , /shop/all , /categories , /categories/:slug , /products/:slug	CatalogStore , ProductCard/Grid/Gallery/VariantSelector , category pages	CatalogApi , ContentApi	products/categories/content: /categories , /products , /products/:slug , /content/home	Catalog store; product state tests
SEARCH-001-005	Search & filters	/search , /shop/all	SearchBar , FilterPanel , ActiveFilters , URL serializer	SearchApi , CatalogApi	/search , /products	URL round-trip; empty/no-result
PDP-001-005	Product detail	/products/:slug	ProductDetailPage , AvailabilityBadge , WhatsAppCta	CatalogApi	/products/:slug	State matrix; ally; SEO meta
CART-001-005	Cart	/cart	CartStore , CartLine , CartPage , merge dialog	CartApi	carts: /cart , /cart/items*	Cart store; stale version; merge conflict
CHK-001-012	Checkout	/checkout/* , /orders/:reference/confirmation	CheckoutStore , step pages, ShippingSummary , CouponField , OrderSummary	CheckoutApi , OrdersApi , ConfigApi	/checkout/shipping-estimate , POST /orders , ◇ payment methods config	Checkout integration; idempotency; error-code UI
ORD-001-009	Orders, tracking, cancel	/orders/:reference , /orders/:reference/tracking , /account/orders	OrderDetailStore , Timeline , StatusBadge	OrdersApi	orders: GET/PATCH /orders/:reference , POST /orders/:reference/cancel , ◇ list , ◇ COD confirm	Status matrix; cancel conflict

PRD requirement group	Frontend feature	Routes	Angular components / stores	API service	Backend module / endpoint	Test coverage
PAY-001-008	Payment proof & review	checkout proof step, order detail, /account/payments, /admin/payments	PaymentProofUploader, ProofViewer, payment panel	PaymentsApi, AdminPaymentsApi	payments: GET /orders/:reference/payment, upload-config, payment-proofs, /admin/payments/:id/*	Upload/retry; re-upload; admin review
SHIP-001-009, PICK-001-004	Fulfillment, shipping, pickup	checkout fulfillment, order detail, /admin/shipping	EstimateVsFinalCost, shipping admin form	CheckoutApi, AdminShippingApi	/checkout/shipping-estimate, /admin/orders/:reference/shipping	Not serviceable; estimate vs final
INV-001-008	Inventory	/admin/inventory, order accept	StockAdjustDialog, inventory table	AdminInventoryApi, AdminOrdersApi	/admin/inventory/:productId/adjust, /admin/orders/:reference/accept	Accept insufficient stock
PRE-001-007	Pre-orders	/products/:slug/pre-order, /account/pre-orders*, /admin/pre-orders	PreOrderForm, pre-order timeline, admin queue	PreOrdersApi, AdminPreOrdersApi	/products/:slug/pre-orders, /pre-orders, /admin/pre-orders/:reference/accept	Eligibility guard; states
SRV-001-008	Student services	/services*, /account/services*, /admin/services*	DynamicServiceForm, ServiceCard, service timeline	ServiceCatalogApi, ServiceRequestsApi, AdminServicesApi	/services, /services/:slug, /services/:slug/requests, /service-requests/:reference	Separation tests; no-file test
QUOTE-001-005	Quotations	/account/services/:reference/quotation, /admin/services/requests/:reference	QuotationCard, QuotationForm	ServiceRequestsApi, AdminServicesApi	/admin/service-requests/:reference/quotes, /service-requests/:reference/quotation/accept	reject`
RET-001-004, REF-001-003	Returns & refunds	/orders/:reference/return, /account/returns/:reference, /admin/returns, /admin/refunds/:refundId	ReturnForm, return timeline, RefundForm	ReturnsApi, AdminReturnsApi	/orders/:reference/returns, /admin/returns/:reference/approve	reject, /admin/refunds/:refundId/complete`

PRD requirement group	Frontend feature	Routes	Angular components / stores	API service	Backend module / endpoint	Test coverage
NOT-001-004, RT-001-003	Notifications & realtime	<code>/account/notifications</code> , <code>/admin/notifications</code> , bell	<code>NotificationStore</code> , <code>RealtimeService</code> , bell	<code>NotificationsApi</code>	notifications + Socket.IO events; ◇ list/read	Dedupe; fallback; reconnect
COUP-001-005	Coupons	checkout coupon field, <code>/admin/coupons</code>	<code>CouponField</code> , coupon admin form	<code>OrdersApi</code> (apply at create), <code>AdminCouponsApi</code>	coupons ◇	Basis-point conversion; invalid coupon
ADM-001-003, RBAC-001-004	Admin shell & RBAC	<code>/admin/**</code> , <code>/403</code>	<code>AdminShell</code> , <code>adminMatch</code> , <code>permissionGuard</code>	<code>MeApi</code>	<code>/me</code> , role/permission (◇ shape)	CanMatch; 403 handling
CMS-001-004	Content & merchandising	<code>/</code> , <code>/admin/content</code>	home modules, <code>ContentEditor</code>	<code>ContentApi</code> , <code>AdminContentApi</code>	<code>/content/home</code> , ◇ admin content	Date window display; no taxonomy change
REP-001-002	Reports	<code>/admin/reports</code>	report views	<code>AdminReportsApi</code>	<code>/admin/reports/:report</code>	Dimension rendering
WA-001-003	WhatsApp support	global header/footer, product, cart, checkout, order, service, admin	<code>WhatsAppCta</code> , <code>ContactBlock</code>	<code>ConfigApi</code>	◇ WhatsApp link data	Hidden w/o config; no secrets in URL
Audit (PAY-007, RBAC)	Audit log view	<code>/admin/audit</code>	audit table	<code>AdminAuditApi</code>	<code>/admin/audit-logs</code>	Read-only
SEO (PRD §15)	SEO	public routes	<code>SeoService</code> , JSON-LD	—	—	Meta/noindex tests
Analytics (PRD §16, UX §22)	Event facade	n/a	<code>AnalyticsService</code>	—	business events via backend	Event contract tests

41. Frontend Open Decisions / Blockers

Format per item: ID · issue · affected features · why it matters · current safe frontend behavior · what needs an owner decision. OPEN DECISION — DO NOT INVENT applies to every row.

41.1 Frontend-specific decisions

ID	Issue	Affected	Why it matters	Safe behavior now	Needs decision
FB-01	Styling engine: this brief says SCSS; the Design System, UX spec, and DESIGN.md say Tailwind; the <code>design-website</code> mockups' markup style is unknown to this plan	all UI	Two styling systems must not coexist; mockup markup may be Tailwind-based	SCSS + CSS variables per this brief (tokens identical to Design System); reproduce the <i>visual result</i> , not the mockup markup; FRONTEND BLOCKER for Phase 1 only until the team confirms	Tech lead: confirm SCSS-only, or approve Tailwind mapped to the same tokens
FB-02	SSR / prerender / SPA	SEO, social previews, hosting	Social crawlers do not run JS; hosting must run Node for SSR	Build SSR-safe; meta via <code>SeoService</code> ; treat previews as unverified	Owner + tech lead
FB-03	Design files not available to the plan author (<code>design-website/</code>)	all screens	brief requires exact reproduction	Page specs from IA/UX/DS; Phase 1 audit amends them	Provide access / confirm audit owner
FB-04	When screenshots reach the backend for non-COD orders (at checkout vs after acceptance) and exact <code>POST /orders</code> proof fields	checkout, payments	sources describe both	\$25.2 interim (build step; send what the contract accepts; else upload on order page)	Backend contract confirmation
FB-05	Email verification gating scope (OD-02)	register, account, checkout	which actions need verified email	No client-side gating; show banner; surface backend refusals	Owner
FB-06	Final payment-method list (OD-20) incl. cash at library	checkout, payments, admin settings	PRD six-method vs brief four-method list	Render only what config/backend returns; cash options equal visual weight; no hard-coded list	Owner
FB-07	Return/refund policy (OD-15/16): periods, non-returnables, evidence, refund SLA	returns, order, product, copy	promises must not be invented	Neutral wording; eligibility from backend flags; no window/SLA text	Owner
FB-08	Whether COD applies to services (OD-14) and reuse of the payment/proof model for quotations	service payment	determines methods shown	Show methods the backend returns for that payment; COD only if listed	Owner
FB-09	Guest follow-up of service requests and quotations (no guest route exists in the IA)	services	a guest cannot accept a quote	No guest quotation route; sign-in path shown	Owner/product

ID	Issue	Affected	Why it matters	Safe behavior now	Needs decision
FB-10	Guest follow-up of pre-orders	pre-orders	IA marks pre-order request conditional	Show reference; sign-in to follow	Owner/product
FB-11	Post-acceptance non-financial order edits (ORD-009 / backend OD-24); exact pending-review editable fields	order detail	what Edit offers	Cancel only; Edit only for backend-declared fields	Owner + operations
FB-12	Palette/fonts vs approved logo; standalone logo file (Q-03)	all UI	tokens are proposals	Tokens centralized; logo asset placeholder; do not recolor logo	Owner supplies logo
FB-13	Numeral style (Arabic-Indic vs Latin) for prices/dates; wording of simple order stages (Q-04, Q-11); hero copy (Q-12)	formatting, copy	consistency	Locale default Intl numerals; stage labels as translation keys; hero copy placeholder-free (neutral)	Owner
FB-14	English URL/SEO strategy; indexing of /search	SEO, i18n	no locale in the IA	No locale in URL; English not independently indexable; /search noindex	Owner
FB-15	Sort options (Q-08) and facet/filter value sources	catalog	avoid invented ranking	Sort control only if backend supplies options; facets from API	Owner + backend
FB-16	Guest token persistence (sessionStorage vs longer)	guest order access	usability vs XSS risk	sessionStorage + "save your order link"	Owner + security review
FB-17	Guest return requests and guest-to-account order merge UI	returns, account	endpoints/policies not specified	Return request for customers only; no merge UI	Owner/backend
FB-18	Missing endpoints/shapes listed in Gap B	many	cannot code safely without them	Build against interfaces + mocks; raise backend tickets	Backend owner (before the phase that needs them)
FB-19	Related products (Q-06)	product detail	IA defines no logic	Not built	Owner
FB-20	Chart library for reports	admin reports	avoid unneeded dependency	tables first; minimal SVG	Tech lead later
FB-21	Related Azhar sub-entry under Books (Q-07) and exact category tree	navigation	brand differentiator vs approved taxonomy	Navigation from /categories data only; Azhar via filters	Owner

ID	Issue	Affected	Why it matters	Safe behavior now	Needs decision
FB-22	Real-time room joining and socket auth refresh details	realtime	verify contract	Revalidate on reconnect; poll fallback	Backend

41.2 Inherited source decisions that affect the frontend

Source ID	Issue	Affected	Safe behavior now
OD-01	Final phone/WhatsApp numbers	header, footer, contact, WhatsApp CTAs	CTAs/phone hidden when config empty; never invented
Q-02	Landmark wording ("Al-Rahbat School" vs "Sisters' School")	footer, contact, pickup info	Render from config only; do not hard-code
OD-03-06	Shipping rate source, final-cost binding, delivery ranges, excluded areas	checkout, order, shipping admin	estimate vs final shown separately; no time promises; no rate table; serviceability message from backend
OD-07	COD non-confirmation timeout	order detail	state stays; no expiry shown
OD-08-09, OD-21	Stock contention, thresholds, partial acceptance	admin accept, inventory	show backend result; no low-stock alerts; no partial acceptance UI
OD-10-11	Pre-order price honoring, allocation	pre-orders	neutral labels, no promises
OD-12-13	Service fields/documents; turnaround	service forms/pages	fields only from backend; empty-safe; no turnaround
OD-17	Coupon governance	coupon field, admin coupons	only exposed fields; no stacking/expiry copy
OD-18	KPI targets	reports	actual data only
OD-19 (IA/UX)	Accessibility/performance/availability targets	all	build to WCAG 2.2 AA; baseline measure only
Backend OD-19	Operational notification/email policy details	notifications	UI renders what backend sends
Backend OD-22	Payment-proof retention/deletion	privacy	no deletion UI; no retention claims
Backend OD-23	Backup RPO/RTO	n/a (ops)	none
Master Brief	Exact service pricing models, non-returnable products, paid advertising	services, returns	not displayed

42. Final Recommended Implementation Sequence

1. Week-zero decisions (blockers): secure the OpenAPI/Postman files and `design-website/`; settle FB-01, FB-02, FB-12; get answers for the Gap B contract items needed in Phases 2-5 (guest cart/session transport, guest order token transport, payment-methods/config endpoint, cart merge trigger, upload-config shape, catalog pagination/sort/facets).
2. Phase 0 → 1 → 2 strictly in order: foundation, design primitives, core API layer.
3. Phase 3 authentication (needed by almost everything).
4. Phases 4 → 5 → 6 storefront, cart/checkout, orders/customer. This is the revenue path; ship it to staging early.
5. Phase 7 services (customer side) once payment components from Phase 6 exist.
6. Phase 8 notifications and Socket.IO (after stores exist to revalidate).
7. Phases 9 → 10 → 11 → 12 Admin: foundation, commerce, payments/services/pre-orders/returns, then reports/content/coupons/settings. Admin can begin alongside Phase 5-7 with a second developer after

Phase 3.

8. Phase 13 SEO/a11y/perf hardening, including the FB-02 implementation.
9. Phase 14 workflow tests on staging, security pass, deployment alignment with the backend's same-origin strategy, owner sign-off on every interim behavior in §41.

Build order inside each page: models/mappers → API service → store → page skeleton with states → real data → forms/mutations → responsive/RTL polish → a11y → tests → traceability entry.

Final consistency check (against the review list in the brief): every major PRD domain has a frontend feature (§40); documented IA routes are covered in §9 (one added, `/admin/audit`, justified by an existing backend endpoint and the requested admin scope); all ten product flows are documented (Flows 1-2, 4, 7 in §23-24; 3 and 8 in §25; 5 in §26; 6 in §27; 9 in §28; 10 in §20.4); auth, RBAC, guest checkout, proof workflow, service/product separation, pre-orders, returns/refunds, notifications, socket-failure fallback, RTL/LTR, SEO, accessibility, responsive, states, and testing are covered; open decisions are preserved in §41; no business rule was invented; the backend remains authoritative for state and authorization; the `design-website/` files are treated as the visual source of truth but could not be inspected by the author (Gap A / FB-03).

End of document.